

This notebook was created by [Jean de Dieu Nyandwi](#) for the love of machine learning community. For any feedback, errors or suggestion, he can be reached on email ([johnjw7084 at gmail dot com](#)), [Twitter](#), or [LinkedIn](#).

Neural Networks for Regression with TensorFlow

Contents

- [1. Intro to Regression with TensorFlow](#)
- [2. Starting Simple: Fitting a Straight Line](#)
 - [2.1 Gathering the data](#)
 - [2.2 Looking in the data](#)
 - [2.3 Preparing the data for a model](#)
 - [2.4 Creating, Compiling and Training a model](#)
 - [2.5 Evaluating a model](#)
 - [2.6 Improving a model](#)
- [3. Going Beyond: A Real world dataset](#)
 - [3.1 Gathering the data](#)
 - [3.2 Looking in the data](#)
 - [3.3 Preparing the data for a model](#)
 - [3.4 Creating, Compiling and Training a model](#)
 - [3.5 Evaluating a model](#)
 - [3.6 Improving a model](#)
 - [3.7 Saving a model](#)
 - [3.8 Final notes](#)

Intro to Regression with TensorFlow

We know neural networks in taking moonshots but they can also be used for regression problems.

In regression task/problems, we are interested in predicting the a single or multiple continous numbers.

Take an example of house price prediction. We can be given the properties/features of the house such as size, region, and number of bedrooms to predict the price of such house. This example is a single number prediction, also termed as **univariate regression**.

Another example appears in object detection(recognizing & localizing image). In order to localize the object with the bounding box, you have got to find the coordinates of the object's center and

the bounding box. The prediction of these coordinates is an example of predicting multiple values at once. It is usually termed as **multivariate regression**.

Any typical architecture for a regression neural network will have common values or ranges of values of **hyperparameters** (hyperparameters are all parameters that you as engineer has to set, such as learning rate, number of layers, etc..). Let's discuss them.

Input, hidden, and Output Layers

The input layer usually has the neurons(or units) equivalent to the number of input features. For example, if our house dataset has 10 variables, 9 input training features, 1 target feature(price of house), then the input neurons will be 9.

The number of hidden layers depend on the problem and the size of the dataset, but generally, it will be between 1 and 5. Same is true about the number of neurons in each hidden layer, it will depend on the problem & dataset size, but generally, neurons can be between 10 to 100.

The number of neurons in output layer will depend on the problem. If you are predicting a single number, it will be 1. If you are predicting the coordinates of the object's center and bounding box during object detection, it will be 4 (because there are 4 coordinates, 2 for object's center, 2 for height/width of the box).

Activation Function

The choice of activation function depends on the problem, but in most cases, **relu** will work well in hidden layers.

The activation function in the output layer is very specific on the goal of the problem. Unlike neural network classifiers that usually use **sigmoid** or **softmax**, regressor doesn't need to have an activation since you want the output values as they are. That being said, you may want to prevent the negative values in the output layer. In that case, you can use **ReLU**.

Training Loss Function

The loss function used in regression is usually **Mean Squared Error(MSE)**. When you are aware that your dataset contain outliers, you can use **Mean Absolute Error (MAE)**. MAE can potentially be used in time series prediction since that type of data tends to have outliers.

Another loss function that is used alot if **Huber loss**. It is combination of both MSE and MAE.

Optimizer

A good rule of thumb when choosing an optimizer is to start with **Adam**. There are other optimizers that you can try, such as SGD(Stockastic Gradient Descent), RMSProp, Nadam, etc...Learn more about them on [TensorFlow optimizer documentation page](#).

Below is a summary of hyerparameter best practices in neural network regressors.

Hyperparameter	Typical value
Neurons at input layer	1 neuron per feature
No of hidden layer(s)	depend on problem, start from 1 to 10

Hyperparameter	Typical value
Neurons per hidden layer	depend on problem, generally 10 to 100
Neurons at input layer	depend on the desired result, 1 for univariate regression
Activation in hidden layers	Relu or its variants(LeakyReLU, SeLU
Activation in output layer	None in most cases
Loss function	MSE or MAE
Optimizer	SGD, Adam, RMSProp

Table: Typical values of hyperparameters in neural network regressors

2. Starting Simple: Fitting a Straight Line

There are many hyperparameters in neural networks and finding the best values of each and each can be overwhelming.

When choosing hyperparameters, it is advised to use hyperparameter tuning tools such as [Keras Tuner](#) to search the best hyperparameters whenever possible. It is nearly impossible to assume that a given value of hyperparameter will work well at first. We usually have to experiment with different values, and a good tool can help you do that quickly.

Let's put all of the above into practice, starting simple, and later, taking a further step into real world dataset.

One of the simplest things we can model is perhaps a linear equation(it has been proven that [neural networks can model any mathematical function](#)).

So, a linear equation forms a straight line. Its form is $y=aX+b$ where a is coefficient (or **weight**) and b is intercept (or **bias**).

Let's assume that we have this equation $y=2X+1$ and we are interested in using neural networks to predict y given any value of X .

But we will start with creating our data based off such equation.

2.1 Gathering the data

Let's first import the libraries. I will import **TensorFlow**, **NumPy** and **Matplotlib** for plotting the straight line.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

After we have imported all relevant libraries, it's time to create our data. We only have one input feature (X) and output label y.

We can either create it with `tf.constant()` or `np.array()`. Both will work, but for convenience, let's use `tf.constant()`.

```
# Create input feature X
X = tf.constant([-2, -1, 0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 10, 12])

# Create label y
y = tf.constant([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0, 9.0, 11.0, 13.0, 15.0, 17.0, 21.0, 25.0])
```

2.2 Looking in the Data

It's always good to look in the data. This can be done in many ways and it depends on the kind of the dataset. If you're working with images, you might want to go through some images and their labels looking if there are no mislabelling.

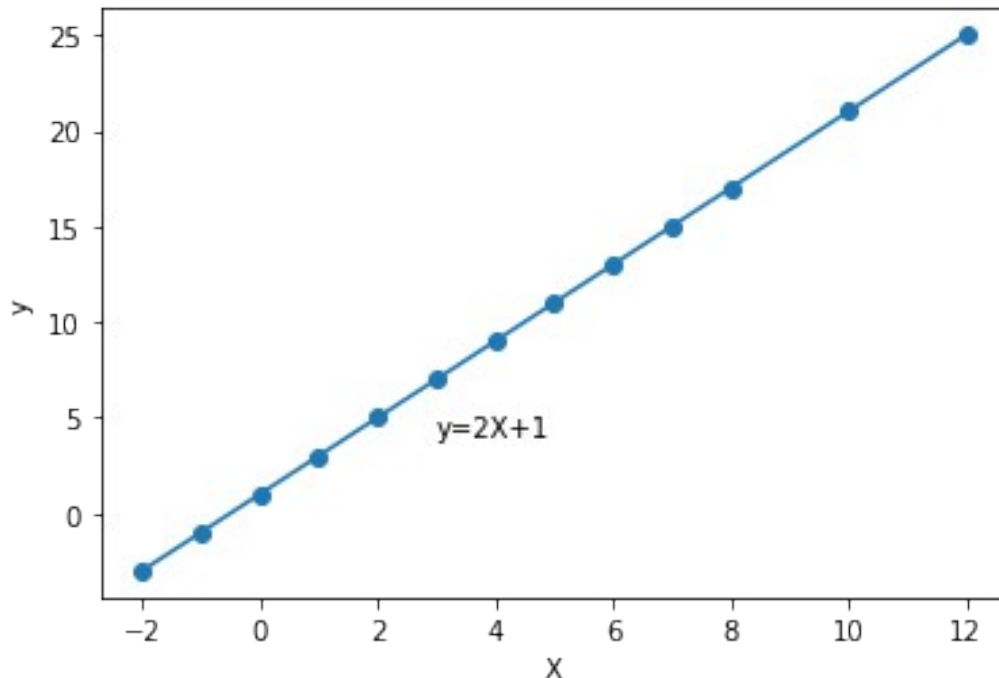
In structured data, or data in tabular form, you might have to visualize the distribution of individual features or plot their relationship.

For us now, we have a simple data. Let's scatter plot X and y.

```
# Visualizing X and y

plt.plot(X,y) #This will plot a line
plt.scatter(X,y)
plt.annotate('y=2X+1', xy=(3,4))
plt.xlabel('X')
plt.ylabel('y')

Text(0, 0.5, 'y')
```



2.3 Preparing data for the model

Usually when working with real world datasets, we need to spend an enormous amount of time preparing it. The type of things to be done depend on the kind of the dataset, but typically, it can be removing/filling missing values, scaling the features either with normalization or standardization, and so on.

For now, we can leave our data as it is. In the next labs, we will see some real world datasets that needs extra work before feeding them to the machine learning model.

2.4 Creating, Compiling and Training a Model

We are going to create the model having a one layer, one neuron(or unit) and as the input data is a single number, we will set the `input_shape` to `[1]`.

Later, we will explain everything we did.

```
model = tf.keras.Sequential([  
    keras.layers.Dense(units=1,  
input_shape=[1])  
])
```

And we can see the model summary. Model summary is essential for quick review of the model architecture. As you can see, we only have one dense layer, and 2 parameters(weight and bias).

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 1)	2
Total params: 2		
Trainable params: 2		
Non-trainable params: 0		

The model that we created is called a Sequential model. We create it by adding one layer after another. If we had many layers, it would be like a sequence or series of layers, one after another, from the input to the output. You can learn more about Sequential API [here](#).

After we have created the model, all we have is an empty graph. We need to do two more things, compiling and training/fitting the model to the presented data.

```
model.compile(optimizer='sgd',  
              loss='mean_squared_error',  
              metrics=['mse'])
```

The single most reason of why we compile the model is to specify the optimizer, loss function and the metrics that we want to track during training.

During training, loss function will be used to measure the difference between the prediction and the actual output. Such difference is called **error**, but we want to measure the **mean of squared error**, hence the name.

Error = Actual value – Predicted value.

On the other hand, optimizer is used to reduce the error between the actual output and predicted value. The optimizer will make continuous guesses until the minimum error is reached. There are many optimizers, but for now let's use SGD (Stochastic Gradient Descent). In later labs, we will explore other optimizers.

By fitting the model to the data, here are what happen:

- The model iterate through each input data point and estimate prediction
- The difference between the actual and predicted value or error is calculated by loss function
- The error is minimized by optimizer as we go through the data
- The above iteration continue until the number of epochs are reached.

Let's see that in action, fitting the model to the data, calculating and reducing the error until we make 500 turns.

```
history = model.fit(X,y, epochs=500)
```

```
Epoch 1/500
1/1 [=====] - 1s 583ms/step - loss: 240.2002
- mse: 240.2002
Epoch 2/500
1/1 [=====] - 0s 7ms/step - loss: 20.7756 -
mse: 20.7756
Epoch 3/500
1/1 [=====] - 0s 7ms/step - loss: 1.9780 -
mse: 1.9780
Epoch 4/500
1/1 [=====] - 0s 10ms/step - loss: 0.3642 -
mse: 0.3642
Epoch 5/500
1/1 [=====] - 0s 12ms/step - loss: 0.2222 -
mse: 0.2222
Epoch 6/500
1/1 [=====] - 0s 8ms/step - loss: 0.2064 -
mse: 0.2064
Epoch 7/500
1/1 [=====] - 0s 8ms/step - loss: 0.2015 -
mse: 0.2015
Epoch 8/500
1/1 [=====] - 0s 8ms/step - loss: 0.1976 -
mse: 0.1976
Epoch 9/500
1/1 [=====] - 0s 11ms/step - loss: 0.1938 -
mse: 0.1938
Epoch 10/500
1/1 [=====] - 0s 5ms/step - loss: 0.1901 -
mse: 0.1901
Epoch 11/500
1/1 [=====] - 0s 7ms/step - loss: 0.1865 -
mse: 0.1865
Epoch 12/500
1/1 [=====] - 0s 6ms/step - loss: 0.1829 -
mse: 0.1829
Epoch 13/500
1/1 [=====] - 0s 10ms/step - loss: 0.1794 -
mse: 0.1794
Epoch 14/500
1/1 [=====] - 0s 12ms/step - loss: 0.1760 -
mse: 0.1760
Epoch 15/500
1/1 [=====] - 0s 11ms/step - loss: 0.1726 -
mse: 0.1726
Epoch 16/500
1/1 [=====] - 0s 7ms/step - loss: 0.1693 -
mse: 0.1693
Epoch 17/500
1/1 [=====] - 0s 6ms/step - loss: 0.1661 -
```

```
mse: 0.1661
Epoch 18/500
1/1 [=====] - 0s 7ms/step - loss: 0.1630 -
mse: 0.1630
Epoch 19/500
1/1 [=====] - 0s 7ms/step - loss: 0.1598 -
mse: 0.1598
Epoch 20/500
1/1 [=====] - 0s 7ms/step - loss: 0.1568 -
mse: 0.1568
Epoch 21/500
1/1 [=====] - 0s 8ms/step - loss: 0.1538 -
mse: 0.1538
Epoch 22/500
1/1 [=====] - 0s 9ms/step - loss: 0.1509 -
mse: 0.1509
Epoch 23/500
1/1 [=====] - 0s 8ms/step - loss: 0.1480 -
mse: 0.1480
Epoch 24/500
1/1 [=====] - 0s 9ms/step - loss: 0.1452 -
mse: 0.1452
Epoch 25/500
1/1 [=====] - 0s 8ms/step - loss: 0.1424 -
mse: 0.1424
Epoch 26/500
1/1 [=====] - 0s 11ms/step - loss: 0.1397 -
mse: 0.1397
Epoch 27/500
1/1 [=====] - 0s 7ms/step - loss: 0.1370 -
mse: 0.1370
Epoch 28/500
1/1 [=====] - 0s 12ms/step - loss: 0.1344 -
mse: 0.1344
Epoch 29/500
1/1 [=====] - 0s 7ms/step - loss: 0.1318 -
mse: 0.1318
Epoch 30/500
1/1 [=====] - 0s 11ms/step - loss: 0.1293 -
mse: 0.1293
Epoch 31/500
1/1 [=====] - 0s 9ms/step - loss: 0.1269 -
mse: 0.1269
Epoch 32/500
1/1 [=====] - 0s 9ms/step - loss: 0.1244 -
mse: 0.1244
Epoch 33/500
1/1 [=====] - 0s 5ms/step - loss: 0.1221 -
mse: 0.1221
Epoch 34/500
```



```
1/1 [=====] - 0s 6ms/step - loss: 0.1197 -  
mse: 0.1197  
Epoch 35/500  
1/1 [=====] - 0s 7ms/step - loss: 0.1175 -  
mse: 0.1175  
Epoch 36/500  
1/1 [=====] - 0s 7ms/step - loss: 0.1152 -  
mse: 0.1152  
Epoch 37/500  
1/1 [=====] - 0s 5ms/step - loss: 0.1130 -  
mse: 0.1130  
Epoch 38/500  
1/1 [=====] - 0s 10ms/step - loss: 0.1109 -  
mse: 0.1109  
Epoch 39/500  
1/1 [=====] - 0s 5ms/step - loss: 0.1087 -  
mse: 0.1087  
Epoch 40/500  
1/1 [=====] - 0s 8ms/step - loss: 0.1067 -  
mse: 0.1067  
Epoch 41/500  
1/1 [=====] - 0s 20ms/step - loss: 0.1046 -  
mse: 0.1046  
Epoch 42/500  
1/1 [=====] - 0s 6ms/step - loss: 0.1026 -  
mse: 0.1026  
Epoch 43/500  
1/1 [=====] - 0s 6ms/step - loss: 0.1007 -  
mse: 0.1007  
Epoch 44/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0988 -  
mse: 0.0988  
Epoch 45/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0969 -  
mse: 0.0969  
Epoch 46/500  
1/1 [=====] - 0s 19ms/step - loss: 0.0950 -  
mse: 0.0950  
Epoch 47/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0932 -  
mse: 0.0932  
Epoch 48/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0914 -  
mse: 0.0914  
Epoch 49/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0897 -  
mse: 0.0897  
Epoch 50/500  
1/1 [=====] - 0s 9ms/step - loss: 0.0880 -  
mse: 0.0880
```

```
Epoch 51/500
1/1 [=====] - 0s 6ms/step - loss: 0.0863 -
mse: 0.0863
Epoch 52/500
1/1 [=====] - 0s 11ms/step - loss: 0.0847 -
mse: 0.0847
Epoch 53/500
1/1 [=====] - 0s 5ms/step - loss: 0.0830 -
mse: 0.0830
Epoch 54/500
1/1 [=====] - 0s 4ms/step - loss: 0.0815 -
mse: 0.0815
Epoch 55/500
1/1 [=====] - 0s 5ms/step - loss: 0.0799 -
mse: 0.0799
Epoch 56/500
1/1 [=====] - 0s 4ms/step - loss: 0.0784 -
mse: 0.0784
Epoch 57/500
1/1 [=====] - 0s 9ms/step - loss: 0.0769 -
mse: 0.0769
Epoch 58/500
1/1 [=====] - 0s 12ms/step - loss: 0.0754 -
mse: 0.0754
Epoch 59/500
1/1 [=====] - 0s 4ms/step - loss: 0.0740 -
mse: 0.0740
Epoch 60/500
1/1 [=====] - 0s 5ms/step - loss: 0.0726 -
mse: 0.0726
Epoch 61/500
1/1 [=====] - 0s 5ms/step - loss: 0.0712 -
mse: 0.0712
Epoch 62/500
1/1 [=====] - 0s 6ms/step - loss: 0.0698 -
mse: 0.0698
Epoch 63/500
1/1 [=====] - 0s 5ms/step - loss: 0.0685 -
mse: 0.0685
Epoch 64/500
1/1 [=====] - 0s 4ms/step - loss: 0.0672 -
mse: 0.0672
Epoch 65/500
1/1 [=====] - 0s 5ms/step - loss: 0.0659 -
mse: 0.0659
Epoch 66/500
1/1 [=====] - 0s 6ms/step - loss: 0.0647 -
mse: 0.0647
Epoch 67/500
1/1 [=====] - 0s 4ms/step - loss: 0.0634 -
```

```
mse: 0.0634
Epoch 68/500
1/1 [=====] - 0s 5ms/step - loss: 0.0622 -
mse: 0.0622
Epoch 69/500
1/1 [=====] - 0s 4ms/step - loss: 0.0610 -
mse: 0.0610
Epoch 70/500
1/1 [=====] - 0s 4ms/step - loss: 0.0599 -
mse: 0.0599
Epoch 71/500
1/1 [=====] - 0s 5ms/step - loss: 0.0587 -
mse: 0.0587
Epoch 72/500
1/1 [=====] - 0s 7ms/step - loss: 0.0576 -
mse: 0.0576
Epoch 73/500
1/1 [=====] - 0s 6ms/step - loss: 0.0565 -
mse: 0.0565
Epoch 74/500
1/1 [=====] - 0s 4ms/step - loss: 0.0554 -
mse: 0.0554
Epoch 75/500
1/1 [=====] - 0s 5ms/step - loss: 0.0544 -
mse: 0.0544
Epoch 76/500
1/1 [=====] - 0s 6ms/step - loss: 0.0533 -
mse: 0.0533
Epoch 77/500
1/1 [=====] - 0s 6ms/step - loss: 0.0523 -
mse: 0.0523
Epoch 78/500
1/1 [=====] - 0s 4ms/step - loss: 0.0513 -
mse: 0.0513
Epoch 79/500
1/1 [=====] - 0s 6ms/step - loss: 0.0503 -
mse: 0.0503
Epoch 80/500
1/1 [=====] - 0s 5ms/step - loss: 0.0494 -
mse: 0.0494
Epoch 81/500
1/1 [=====] - 0s 4ms/step - loss: 0.0484 -
mse: 0.0484
Epoch 82/500
1/1 [=====] - 0s 4ms/step - loss: 0.0475 -
mse: 0.0475
Epoch 83/500
1/1 [=====] - 0s 4ms/step - loss: 0.0466 -
mse: 0.0466
Epoch 84/500
```

```
1/1 [=====] - 0s 5ms/step - loss: 0.0457 -  
mse: 0.0457  
Epoch 85/500  
1/1 [=====] - 0s 4ms/step - loss: 0.0448 -  
mse: 0.0448  
Epoch 86/500  
1/1 [=====] - 0s 13ms/step - loss: 0.0440 -  
mse: 0.0440  
Epoch 87/500  
1/1 [=====] - 0s 10ms/step - loss: 0.0431 -  
mse: 0.0431  
Epoch 88/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0423 -  
mse: 0.0423  
Epoch 89/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0415 -  
mse: 0.0415  
Epoch 90/500  
1/1 [=====] - 0s 10ms/step - loss: 0.0407 -  
mse: 0.0407  
Epoch 91/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0399 -  
mse: 0.0399  
Epoch 92/500  
1/1 [=====] - 0s 14ms/step - loss: 0.0392 -  
mse: 0.0392  
Epoch 93/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0384 -  
mse: 0.0384  
Epoch 94/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0377 -  
mse: 0.0377  
Epoch 95/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0370 -  
mse: 0.0370  
Epoch 96/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0363 -  
mse: 0.0363  
Epoch 97/500  
1/1 [=====] - 0s 16ms/step - loss: 0.0356 -  
mse: 0.0356  
Epoch 98/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0349 -  
mse: 0.0349  
Epoch 99/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0342 -  
mse: 0.0342  
Epoch 100/500  
1/1 [=====] - 0s 10ms/step - loss: 0.0336 -  
mse: 0.0336
```

```
Epoch 101/500
1/1 [=====] - 0s 7ms/step - loss: 0.0329 -
mse: 0.0329
Epoch 102/500
1/1 [=====] - 0s 6ms/step - loss: 0.0323 -
mse: 0.0323
Epoch 103/500
1/1 [=====] - 0s 10ms/step - loss: 0.0317 -
mse: 0.0317
Epoch 104/500
1/1 [=====] - 0s 5ms/step - loss: 0.0311 -
mse: 0.0311
Epoch 105/500
1/1 [=====] - 0s 9ms/step - loss: 0.0305 -
mse: 0.0305
Epoch 106/500
1/1 [=====] - 0s 12ms/step - loss: 0.0299 -
mse: 0.0299
Epoch 107/500
1/1 [=====] - 0s 8ms/step - loss: 0.0294 -
mse: 0.0294
Epoch 108/500
1/1 [=====] - 0s 9ms/step - loss: 0.0288 -
mse: 0.0288
Epoch 109/500
1/1 [=====] - 0s 8ms/step - loss: 0.0282 -
mse: 0.0282
Epoch 110/500
1/1 [=====] - 0s 5ms/step - loss: 0.0277 -
mse: 0.0277
Epoch 111/500
1/1 [=====] - 0s 10ms/step - loss: 0.0272 -
mse: 0.0272
Epoch 112/500
1/1 [=====] - 0s 11ms/step - loss: 0.0267 -
mse: 0.0267
Epoch 113/500
1/1 [=====] - 0s 5ms/step - loss: 0.0261 -
mse: 0.0261
Epoch 114/500
1/1 [=====] - 0s 5ms/step - loss: 0.0257 -
mse: 0.0257
Epoch 115/500
1/1 [=====] - 0s 6ms/step - loss: 0.0252 -
mse: 0.0252
Epoch 116/500
1/1 [=====] - 0s 11ms/step - loss: 0.0247 -
mse: 0.0247
Epoch 117/500
1/1 [=====] - 0s 8ms/step - loss: 0.0242 -
```

```
mse: 0.0242
Epoch 118/500
1/1 [=====] - 0s 9ms/step - loss: 0.0237 -
mse: 0.0237
Epoch 119/500
1/1 [=====] - 0s 9ms/step - loss: 0.0233 -
mse: 0.0233
Epoch 120/500
1/1 [=====] - 0s 9ms/step - loss: 0.0229 -
mse: 0.0229
Epoch 121/500
1/1 [=====] - 0s 5ms/step - loss: 0.0224 -
mse: 0.0224
Epoch 122/500
1/1 [=====] - 0s 8ms/step - loss: 0.0220 -
mse: 0.0220
Epoch 123/500
1/1 [=====] - 0s 9ms/step - loss: 0.0216 -
mse: 0.0216
Epoch 124/500
1/1 [=====] - 0s 9ms/step - loss: 0.0212 -
mse: 0.0212
Epoch 125/500
1/1 [=====] - 0s 6ms/step - loss: 0.0208 -
mse: 0.0208
Epoch 126/500
1/1 [=====] - 0s 8ms/step - loss: 0.0204 -
mse: 0.0204
Epoch 127/500
1/1 [=====] - 0s 15ms/step - loss: 0.0200 -
mse: 0.0200
Epoch 128/500
1/1 [=====] - 0s 7ms/step - loss: 0.0196 -
mse: 0.0196
Epoch 129/500
1/1 [=====] - 0s 10ms/step - loss: 0.0192 -
mse: 0.0192
Epoch 130/500
1/1 [=====] - 0s 8ms/step - loss: 0.0188 -
mse: 0.0188
Epoch 131/500
1/1 [=====] - 0s 7ms/step - loss: 0.0185 -
mse: 0.0185
Epoch 132/500
1/1 [=====] - 0s 10ms/step - loss: 0.0181 -
mse: 0.0181
Epoch 133/500
1/1 [=====] - 0s 15ms/step - loss: 0.0178 -
mse: 0.0178
Epoch 134/500
```

```
1/1 [=====] - 0s 7ms/step - loss: 0.0175 -  
mse: 0.0175  
Epoch 135/500  
1/1 [=====] - 0s 13ms/step - loss: 0.0171 -  
mse: 0.0171  
Epoch 136/500  
1/1 [=====] - 0s 11ms/step - loss: 0.0168 -  
mse: 0.0168  
Epoch 137/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0165 -  
mse: 0.0165  
Epoch 138/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0162 -  
mse: 0.0162  
Epoch 139/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0158 -  
mse: 0.0158  
Epoch 140/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0155 -  
mse: 0.0155  
Epoch 141/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0152 -  
mse: 0.0152  
Epoch 142/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0150 -  
mse: 0.0150  
Epoch 143/500  
1/1 [=====] - 0s 15ms/step - loss: 0.0147 -  
mse: 0.0147  
Epoch 144/500  
1/1 [=====] - 0s 10ms/step - loss: 0.0144 -  
mse: 0.0144  
Epoch 145/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0141 -  
mse: 0.0141  
Epoch 146/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0138 -  
mse: 0.0138  
Epoch 147/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0136 -  
mse: 0.0136  
Epoch 148/500  
1/1 [=====] - 0s 11ms/step - loss: 0.0133 -  
mse: 0.0133  
Epoch 149/500  
1/1 [=====] - 0s 4ms/step - loss: 0.0131 -  
mse: 0.0131  
Epoch 150/500  
1/1 [=====] - 0s 13ms/step - loss: 0.0128 -  
mse: 0.0128
```

```
Epoch 151/500
1/1 [=====] - 0s 5ms/step - loss: 0.0126 -
mse: 0.0126
Epoch 152/500
1/1 [=====] - 0s 5ms/step - loss: 0.0123 -
mse: 0.0123
Epoch 153/500
1/1 [=====] - 0s 7ms/step - loss: 0.0121 -
mse: 0.0121
Epoch 154/500
1/1 [=====] - 0s 12ms/step - loss: 0.0119 -
mse: 0.0119
Epoch 155/500
1/1 [=====] - 0s 9ms/step - loss: 0.0116 -
mse: 0.0116
Epoch 156/500
1/1 [=====] - 0s 14ms/step - loss: 0.0114 -
mse: 0.0114
Epoch 157/500
1/1 [=====] - 0s 11ms/step - loss: 0.0112 -
mse: 0.0112
Epoch 158/500
1/1 [=====] - 0s 4ms/step - loss: 0.0110 -
mse: 0.0110
Epoch 159/500
1/1 [=====] - 0s 7ms/step - loss: 0.0108 -
mse: 0.0108
Epoch 160/500
1/1 [=====] - 0s 5ms/step - loss: 0.0106 -
mse: 0.0106
Epoch 161/500
1/1 [=====] - 0s 7ms/step - loss: 0.0104 -
mse: 0.0104
Epoch 162/500
1/1 [=====] - 0s 7ms/step - loss: 0.0102 -
mse: 0.0102
Epoch 163/500
1/1 [=====] - 0s 14ms/step - loss: 0.0100 -
mse: 0.0100
Epoch 164/500
1/1 [=====] - 0s 5ms/step - loss: 0.0098 -
mse: 0.0098
Epoch 165/500
1/1 [=====] - 0s 8ms/step - loss: 0.0096 -
mse: 0.0096
Epoch 166/500
1/1 [=====] - 0s 4ms/step - loss: 0.0094 -
mse: 0.0094
Epoch 167/500
1/1 [=====] - 0s 6ms/step - loss: 0.0092 -
```



```
mse: 0.0092
Epoch 168/500
1/1 [=====] - 0s 8ms/step - loss: 0.0091 -
mse: 0.0091
Epoch 169/500
1/1 [=====] - 0s 5ms/step - loss: 0.0089 -
mse: 0.0089
Epoch 170/500
1/1 [=====] - 0s 5ms/step - loss: 0.0087 -
mse: 0.0087
Epoch 171/500
1/1 [=====] - 0s 7ms/step - loss: 0.0086 -
mse: 0.0086
Epoch 172/500
1/1 [=====] - 0s 12ms/step - loss: 0.0084 -
mse: 0.0084
Epoch 173/500
1/1 [=====] - 0s 17ms/step - loss: 0.0082 -
mse: 0.0082
Epoch 174/500
1/1 [=====] - 0s 5ms/step - loss: 0.0081 -
mse: 0.0081
Epoch 175/500
1/1 [=====] - 0s 5ms/step - loss: 0.0079 -
mse: 0.0079
Epoch 176/500
1/1 [=====] - 0s 10ms/step - loss: 0.0078 -
mse: 0.0078
Epoch 177/500
1/1 [=====] - 0s 5ms/step - loss: 0.0076 -
mse: 0.0076
Epoch 178/500
1/1 [=====] - 0s 8ms/step - loss: 0.0075 -
mse: 0.0075
Epoch 179/500
1/1 [=====] - 0s 7ms/step - loss: 0.0073 -
mse: 0.0073
Epoch 180/500
1/1 [=====] - 0s 4ms/step - loss: 0.0072 -
mse: 0.0072
Epoch 181/500
1/1 [=====] - 0s 8ms/step - loss: 0.0071 -
mse: 0.0071
Epoch 182/500
1/1 [=====] - 0s 5ms/step - loss: 0.0069 -
mse: 0.0069
Epoch 183/500
1/1 [=====] - 0s 6ms/step - loss: 0.0068 -
mse: 0.0068
Epoch 184/500
```

```
1/1 [=====] - 0s 14ms/step - loss: 0.0067 -  
mse: 0.0067  
Epoch 185/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0065 -  
mse: 0.0065  
Epoch 186/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0064 -  
mse: 0.0064  
Epoch 187/500  
1/1 [=====] - 0s 11ms/step - loss: 0.0063 -  
mse: 0.0063  
Epoch 188/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0062 -  
mse: 0.0062  
Epoch 189/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0061 -  
mse: 0.0061  
Epoch 190/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0059 -  
mse: 0.0059  
Epoch 191/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0058 -  
mse: 0.0058  
Epoch 192/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0057 -  
mse: 0.0057  
Epoch 193/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0056 -  
mse: 0.0056  
Epoch 194/500  
1/1 [=====] - 0s 8ms/step - loss: 0.0055 -  
mse: 0.0055  
Epoch 195/500  
1/1 [=====] - 0s 10ms/step - loss: 0.0054 -  
mse: 0.0054  
Epoch 196/500  
1/1 [=====] - 0s 23ms/step - loss: 0.0053 -  
mse: 0.0053  
Epoch 197/500  
1/1 [=====] - 0s 16ms/step - loss: 0.0052 -  
mse: 0.0052  
Epoch 198/500  
1/1 [=====] - 0s 12ms/step - loss: 0.0051 -  
mse: 0.0051  
Epoch 199/500  
1/1 [=====] - 0s 12ms/step - loss: 0.0050 -  
mse: 0.0050  
Epoch 200/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0049 -
```

```
mse: 0.0049
Epoch 201/500
1/1 [=====] - 0s 10ms/step - loss: 0.0048 -
mse: 0.0048
Epoch 202/500
1/1 [=====] - 0s 12ms/step - loss: 0.0047 -
mse: 0.0047
Epoch 203/500
1/1 [=====] - 0s 16ms/step - loss: 0.0046 -
mse: 0.0046
Epoch 204/500
1/1 [=====] - 0s 5ms/step - loss: 0.0045 -
mse: 0.0045
Epoch 205/500
1/1 [=====] - 0s 9ms/step - loss: 0.0044 -
mse: 0.0044
Epoch 206/500
1/1 [=====] - 0s 7ms/step - loss: 0.0044 -
mse: 0.0044
Epoch 207/500
1/1 [=====] - 0s 9ms/step - loss: 0.0043 -
mse: 0.0043
Epoch 208/500
1/1 [=====] - 0s 6ms/step - loss: 0.0042 -
mse: 0.0042
Epoch 209/500
1/1 [=====] - 0s 8ms/step - loss: 0.0041 -
mse: 0.0041
Epoch 210/500
1/1 [=====] - 0s 7ms/step - loss: 0.0040 -
mse: 0.0040
Epoch 211/500
1/1 [=====] - 0s 5ms/step - loss: 0.0040 -
mse: 0.0040
Epoch 212/500
1/1 [=====] - 0s 6ms/step - loss: 0.0039 -
mse: 0.0039
Epoch 213/500
1/1 [=====] - 0s 6ms/step - loss: 0.0038 -
mse: 0.0038
Epoch 214/500
1/1 [=====] - 0s 5ms/step - loss: 0.0037 -
mse: 0.0037
Epoch 215/500
1/1 [=====] - 0s 5ms/step - loss: 0.0037 -
mse: 0.0037
Epoch 216/500
1/1 [=====] - 0s 6ms/step - loss: 0.0036 -
mse: 0.0036
```

```
Epoch 217/500
1/1 [=====] - 0s 6ms/step - loss: 0.0035 -
mse: 0.0035
Epoch 218/500
1/1 [=====] - 0s 11ms/step - loss: 0.0035 -
mse: 0.0035
Epoch 219/500
1/1 [=====] - 0s 6ms/step - loss: 0.0034 -
mse: 0.0034
Epoch 220/500
1/1 [=====] - 0s 5ms/step - loss: 0.0033 -
mse: 0.0033
Epoch 221/500
1/1 [=====] - 0s 9ms/step - loss: 0.0033 -
mse: 0.0033
Epoch 222/500
1/1 [=====] - 0s 6ms/step - loss: 0.0032 -
mse: 0.0032
Epoch 223/500
1/1 [=====] - 0s 13ms/step - loss: 0.0031 -
mse: 0.0031
Epoch 224/500
1/1 [=====] - 0s 6ms/step - loss: 0.0031 -
mse: 0.0031
Epoch 225/500
1/1 [=====] - 0s 18ms/step - loss: 0.0030 -
mse: 0.0030
Epoch 226/500
1/1 [=====] - 0s 8ms/step - loss: 0.0030 -
mse: 0.0030
Epoch 227/500
1/1 [=====] - 0s 8ms/step - loss: 0.0029 -
mse: 0.0029
Epoch 228/500
1/1 [=====] - 0s 23ms/step - loss: 0.0029 -
mse: 0.0029
Epoch 229/500
1/1 [=====] - 0s 9ms/step - loss: 0.0028 -
mse: 0.0028
Epoch 230/500
1/1 [=====] - 0s 6ms/step - loss: 0.0027 -
mse: 0.0027
Epoch 231/500
1/1 [=====] - 0s 6ms/step - loss: 0.0027 -
mse: 0.0027
Epoch 232/500
1/1 [=====] - 0s 7ms/step - loss: 0.0026 -
mse: 0.0026
Epoch 233/500
```

```
1/1 [=====] - 0s 7ms/step - loss: 0.0026 -  
mse: 0.0026  
Epoch 234/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0025 -  
mse: 0.0025  
Epoch 235/500  
1/1 [=====] - 0s 18ms/step - loss: 0.0025 -  
mse: 0.0025  
Epoch 236/500  
1/1 [=====] - 0s 12ms/step - loss: 0.0024 -  
mse: 0.0024  
Epoch 237/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0024 -  
mse: 0.0024  
Epoch 238/500  
1/1 [=====] - 0s 24ms/step - loss: 0.0024 -  
mse: 0.0024  
Epoch 239/500  
1/1 [=====] - 0s 29ms/step - loss: 0.0023 -  
mse: 0.0023  
Epoch 240/500  
1/1 [=====] - 0s 24ms/step - loss: 0.0023 -  
mse: 0.0023  
Epoch 241/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0022 -  
mse: 0.0022  
Epoch 242/500  
1/1 [=====] - 0s 9ms/step - loss: 0.0022 -  
mse: 0.0022  
Epoch 243/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0021 -  
mse: 0.0021  
Epoch 244/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0021 -  
mse: 0.0021  
Epoch 245/500  
1/1 [=====] - 0s 5ms/step - loss: 0.0021 -  
mse: 0.0021  
Epoch 246/500  
1/1 [=====] - 0s 6ms/step - loss: 0.0020 -  
mse: 0.0020  
Epoch 247/500  
1/1 [=====] - 0s 11ms/step - loss: 0.0020 -  
mse: 0.0020  
Epoch 248/500  
1/1 [=====] - 0s 13ms/step - loss: 0.0019 -  
mse: 0.0019  
Epoch 249/500  
1/1 [=====] - 0s 7ms/step - loss: 0.0019 -
```

```
mse: 0.0019
Epoch 250/500
1/1 [=====] - 0s 6ms/step - loss: 0.0019 -
mse: 0.0019
Epoch 251/500
1/1 [=====] - 0s 9ms/step - loss: 0.0018 -
mse: 0.0018
Epoch 252/500
1/1 [=====] - 0s 4ms/step - loss: 0.0018 -
mse: 0.0018
Epoch 253/500
1/1 [=====] - 0s 5ms/step - loss: 0.0018 -
mse: 0.0018
Epoch 254/500
1/1 [=====] - 0s 9ms/step - loss: 0.0017 -
mse: 0.0017
Epoch 255/500
1/1 [=====] - 0s 5ms/step - loss: 0.0017 -
mse: 0.0017
Epoch 256/500
1/1 [=====] - 0s 5ms/step - loss: 0.0017 -
mse: 0.0017
Epoch 257/500
1/1 [=====] - 0s 6ms/step - loss: 0.0016 -
mse: 0.0016
Epoch 258/500
1/1 [=====] - 0s 6ms/step - loss: 0.0016 -
mse: 0.0016
Epoch 259/500
1/1 [=====] - 0s 6ms/step - loss: 0.0016 -
mse: 0.0016
Epoch 260/500
1/1 [=====] - 0s 6ms/step - loss: 0.0015 -
mse: 0.0015
Epoch 261/500
1/1 [=====] - 0s 5ms/step - loss: 0.0015 -
mse: 0.0015
Epoch 262/500
1/1 [=====] - 0s 8ms/step - loss: 0.0015 -
mse: 0.0015
Epoch 263/500
1/1 [=====] - 0s 4ms/step - loss: 0.0015 -
mse: 0.0015
Epoch 264/500
1/1 [=====] - 0s 14ms/step - loss: 0.0014 -
mse: 0.0014
Epoch 265/500
1/1 [=====] - 0s 21ms/step - loss: 0.0014 -
mse: 0.0014
```

```
Epoch 266/500
1/1 [=====] - 0s 30ms/step - loss: 0.0014 -
mse: 0.0014
Epoch 267/500
1/1 [=====] - 0s 6ms/step - loss: 0.0013 -
mse: 0.0013
Epoch 268/500
1/1 [=====] - 0s 8ms/step - loss: 0.0013 -
mse: 0.0013
Epoch 269/500
1/1 [=====] - 0s 7ms/step - loss: 0.0013 -
mse: 0.0013
Epoch 270/500
1/1 [=====] - 0s 8ms/step - loss: 0.0013 -
mse: 0.0013
Epoch 271/500
1/1 [=====] - 0s 8ms/step - loss: 0.0012 -
mse: 0.0012
Epoch 272/500
1/1 [=====] - 0s 5ms/step - loss: 0.0012 -
mse: 0.0012
Epoch 273/500
1/1 [=====] - 0s 8ms/step - loss: 0.0012 -
mse: 0.0012
Epoch 274/500
1/1 [=====] - 0s 5ms/step - loss: 0.0012 -
mse: 0.0012
Epoch 275/500
1/1 [=====] - 0s 6ms/step - loss: 0.0012 -
mse: 0.0012
Epoch 276/500
1/1 [=====] - 0s 5ms/step - loss: 0.0011 -
mse: 0.0011
Epoch 277/500
1/1 [=====] - 0s 17ms/step - loss: 0.0011 -
mse: 0.0011
Epoch 278/500
1/1 [=====] - 0s 6ms/step - loss: 0.0011 -
mse: 0.0011
Epoch 279/500
1/1 [=====] - 0s 5ms/step - loss: 0.0011 -
mse: 0.0011
Epoch 280/500
1/1 [=====] - 0s 5ms/step - loss: 0.0010 -
mse: 0.0010
Epoch 281/500
1/1 [=====] - 0s 13ms/step - loss: 0.0010 -
mse: 0.0010
Epoch 282/500
```

```
1/1 [=====] - 0s 12ms/step - loss: 0.0010 -  
mse: 0.0010  
Epoch 283/500  
1/1 [=====] - 0s 14ms/step - loss: 9.8983e-04  
- mse: 9.8983e-04  
Epoch 284/500  
1/1 [=====] - 0s 5ms/step - loss: 9.7094e-04  
- mse: 9.7094e-04  
Epoch 285/500  
1/1 [=====] - 0s 6ms/step - loss: 9.5243e-04  
- mse: 9.5243e-04  
Epoch 286/500  
1/1 [=====] - 0s 6ms/step - loss: 9.3426e-04  
- mse: 9.3426e-04  
Epoch 287/500  
1/1 [=====] - 0s 17ms/step - loss: 9.1644e-04  
- mse: 9.1644e-04  
Epoch 288/500  
1/1 [=====] - 0s 5ms/step - loss: 8.9895e-04  
- mse: 8.9895e-04  
Epoch 289/500  
1/1 [=====] - 0s 6ms/step - loss: 8.8180e-04  
- mse: 8.8180e-04  
Epoch 290/500  
1/1 [=====] - 0s 5ms/step - loss: 8.6499e-04  
- mse: 8.6499e-04  
Epoch 291/500  
1/1 [=====] - 0s 7ms/step - loss: 8.4849e-04  
- mse: 8.4849e-04  
Epoch 292/500  
1/1 [=====] - 0s 5ms/step - loss: 8.3230e-04  
- mse: 8.3230e-04  
Epoch 293/500  
1/1 [=====] - 0s 6ms/step - loss: 8.1643e-04  
- mse: 8.1643e-04  
Epoch 294/500  
1/1 [=====] - 0s 6ms/step - loss: 8.0086e-04  
- mse: 8.0086e-04  
Epoch 295/500  
1/1 [=====] - 0s 6ms/step - loss: 7.8558e-04  
- mse: 7.8558e-04  
Epoch 296/500  
1/1 [=====] - 0s 6ms/step - loss: 7.7058e-04  
- mse: 7.7058e-04  
Epoch 297/500  
1/1 [=====] - 0s 10ms/step - loss: 7.5588e-04  
- mse: 7.5588e-04  
Epoch 298/500  
1/1 [=====] - 0s 15ms/step - loss: 7.4147e-04
```



```
- mse: 7.4147e-04
Epoch 299/500
1/1 [=====] - 0s 22ms/step - loss: 7.2734e-04
- mse: 7.2734e-04
Epoch 300/500
1/1 [=====] - 0s 9ms/step - loss: 7.1345e-04
- mse: 7.1345e-04
Epoch 301/500
1/1 [=====] - 0s 8ms/step - loss: 6.9984e-04
- mse: 6.9984e-04
Epoch 302/500
1/1 [=====] - 0s 6ms/step - loss: 6.8650e-04
- mse: 6.8650e-04
Epoch 303/500
1/1 [=====] - 0s 8ms/step - loss: 6.7341e-04
- mse: 6.7341e-04
Epoch 304/500
1/1 [=====] - 0s 68ms/step - loss: 6.6056e-04
- mse: 6.6056e-04
Epoch 305/500
1/1 [=====] - 0s 17ms/step - loss: 6.4796e-04
- mse: 6.4796e-04
Epoch 306/500
1/1 [=====] - 0s 6ms/step - loss: 6.3560e-04
- mse: 6.3560e-04
Epoch 307/500
1/1 [=====] - 0s 6ms/step - loss: 6.2348e-04
- mse: 6.2348e-04
Epoch 308/500
1/1 [=====] - 0s 9ms/step - loss: 6.1158e-04
- mse: 6.1158e-04
Epoch 309/500
1/1 [=====] - 0s 12ms/step - loss: 5.9992e-04
- mse: 5.9992e-04
Epoch 310/500
1/1 [=====] - 0s 11ms/step - loss: 5.8847e-04
- mse: 5.8847e-04
Epoch 311/500
1/1 [=====] - 0s 6ms/step - loss: 5.7725e-04
- mse: 5.7725e-04
Epoch 312/500
1/1 [=====] - 0s 8ms/step - loss: 5.6624e-04
- mse: 5.6624e-04
Epoch 313/500
1/1 [=====] - 0s 8ms/step - loss: 5.5544e-04
- mse: 5.5544e-04
Epoch 314/500
1/1 [=====] - 0s 11ms/step - loss: 5.4484e-04
- mse: 5.4484e-04
```

```
Epoch 315/500
1/1 [=====] - 0s 11ms/step - loss: 5.3445e-04
- mse: 5.3445e-04
Epoch 316/500
1/1 [=====] - 0s 12ms/step - loss: 5.2425e-04
- mse: 5.2425e-04
Epoch 317/500
1/1 [=====] - 0s 6ms/step - loss: 5.1425e-04
- mse: 5.1425e-04
Epoch 318/500
1/1 [=====] - 0s 18ms/step - loss: 5.0444e-04
- mse: 5.0444e-04
Epoch 319/500
1/1 [=====] - 0s 18ms/step - loss: 4.9482e-04
- mse: 4.9482e-04
Epoch 320/500
1/1 [=====] - 0s 8ms/step - loss: 4.8538e-04
- mse: 4.8538e-04
Epoch 321/500
1/1 [=====] - 0s 6ms/step - loss: 4.7613e-04
- mse: 4.7613e-04
Epoch 322/500
1/1 [=====] - 0s 7ms/step - loss: 4.6704e-04
- mse: 4.6704e-04
Epoch 323/500
1/1 [=====] - 0s 10ms/step - loss: 4.5813e-04
- mse: 4.5813e-04
Epoch 324/500
1/1 [=====] - 0s 18ms/step - loss: 4.4940e-04
- mse: 4.4940e-04
Epoch 325/500
1/1 [=====] - 0s 11ms/step - loss: 4.4082e-04
- mse: 4.4082e-04
Epoch 326/500
1/1 [=====] - 0s 11ms/step - loss: 4.3242e-04
- mse: 4.3242e-04
Epoch 327/500
1/1 [=====] - 0s 8ms/step - loss: 4.2417e-04
- mse: 4.2417e-04
Epoch 328/500
1/1 [=====] - 0s 10ms/step - loss: 4.1607e-04
- mse: 4.1607e-04
Epoch 329/500
1/1 [=====] - 0s 8ms/step - loss: 4.0814e-04
- mse: 4.0814e-04
Epoch 330/500
1/1 [=====] - 0s 12ms/step - loss: 4.0036e-04
- mse: 4.0036e-04
Epoch 331/500
```

```
1/1 [=====] - 0s 12ms/step - loss: 3.9272e-04
- mse: 3.9272e-04
Epoch 332/500
1/1 [=====] - 0s 9ms/step - loss: 3.8523e-04
- mse: 3.8523e-04
Epoch 333/500
1/1 [=====] - 0s 8ms/step - loss: 3.7788e-04
- mse: 3.7788e-04
Epoch 334/500
1/1 [=====] - 0s 6ms/step - loss: 3.7067e-04
- mse: 3.7067e-04
Epoch 335/500
1/1 [=====] - 0s 9ms/step - loss: 3.6360e-04
- mse: 3.6360e-04
Epoch 336/500
1/1 [=====] - 0s 7ms/step - loss: 3.5666e-04
- mse: 3.5666e-04
Epoch 337/500
1/1 [=====] - 0s 6ms/step - loss: 3.4985e-04
- mse: 3.4985e-04
Epoch 338/500
1/1 [=====] - 0s 10ms/step - loss: 3.4318e-04
- mse: 3.4318e-04
Epoch 339/500
1/1 [=====] - 0s 6ms/step - loss: 3.3664e-04
- mse: 3.3664e-04
Epoch 340/500
1/1 [=====] - 0s 14ms/step - loss: 3.3022e-04
- mse: 3.3022e-04
Epoch 341/500
1/1 [=====] - 0s 6ms/step - loss: 3.2392e-04
- mse: 3.2392e-04
Epoch 342/500
1/1 [=====] - 0s 7ms/step - loss: 3.1774e-04
- mse: 3.1774e-04
Epoch 343/500
1/1 [=====] - 0s 5ms/step - loss: 3.1168e-04
- mse: 3.1168e-04
Epoch 344/500
1/1 [=====] - 0s 8ms/step - loss: 3.0573e-04
- mse: 3.0573e-04
Epoch 345/500
1/1 [=====] - 0s 9ms/step - loss: 2.9990e-04
- mse: 2.9990e-04
Epoch 346/500
1/1 [=====] - 0s 4ms/step - loss: 2.9418e-04
- mse: 2.9418e-04
Epoch 347/500
1/1 [=====] - 0s 16ms/step - loss: 2.8857e-04
- mse: 2.8857e-04
```

Epoch 348/500
1/1 [=====] - 0s 6ms/step - loss: 2.8307e-04
- mse: 2.8307e-04
Epoch 349/500
1/1 [=====] - 0s 12ms/step - loss: 2.7767e-04
- mse: 2.7767e-04
Epoch 350/500
1/1 [=====] - 0s 5ms/step - loss: 2.7237e-04
- mse: 2.7237e-04
Epoch 351/500
1/1 [=====] - 0s 51ms/step - loss: 2.6718e-04
- mse: 2.6718e-04
Epoch 352/500
1/1 [=====] - 0s 9ms/step - loss: 2.6208e-04
- mse: 2.6208e-04
Epoch 353/500
1/1 [=====] - 0s 8ms/step - loss: 2.5708e-04
- mse: 2.5708e-04
Epoch 354/500
1/1 [=====] - 0s 6ms/step - loss: 2.5218e-04
- mse: 2.5218e-04
Epoch 355/500
1/1 [=====] - 0s 4ms/step - loss: 2.4736e-04
- mse: 2.4736e-04
Epoch 356/500
1/1 [=====] - 0s 8ms/step - loss: 2.4264e-04
- mse: 2.4264e-04
Epoch 357/500
1/1 [=====] - 0s 14ms/step - loss: 2.3802e-04
- mse: 2.3802e-04
Epoch 358/500
1/1 [=====] - 0s 7ms/step - loss: 2.3348e-04
- mse: 2.3348e-04
Epoch 359/500
1/1 [=====] - 0s 9ms/step - loss: 2.2903e-04
- mse: 2.2903e-04
Epoch 360/500
1/1 [=====] - 0s 7ms/step - loss: 2.2465e-04
- mse: 2.2465e-04
Epoch 361/500
1/1 [=====] - 0s 15ms/step - loss: 2.2037e-04
- mse: 2.2037e-04
Epoch 362/500
1/1 [=====] - 0s 10ms/step - loss: 2.1617e-04
- mse: 2.1617e-04
Epoch 363/500
1/1 [=====] - 0s 8ms/step - loss: 2.1205e-04
- mse: 2.1205e-04
Epoch 364/500

```
1/1 [=====] - 0s 7ms/step - loss: 2.0800e-04
- mse: 2.0800e-04
Epoch 365/500
1/1 [=====] - 0s 13ms/step - loss: 2.0403e-04
- mse: 2.0403e-04
Epoch 366/500
1/1 [=====] - 0s 10ms/step - loss: 2.0014e-04
- mse: 2.0014e-04
Epoch 367/500
1/1 [=====] - 0s 7ms/step - loss: 1.9632e-04
- mse: 1.9632e-04
Epoch 368/500
1/1 [=====] - 0s 19ms/step - loss: 1.9258e-04
- mse: 1.9258e-04
Epoch 369/500
1/1 [=====] - 0s 13ms/step - loss: 1.8891e-04
- mse: 1.8891e-04
Epoch 370/500
1/1 [=====] - 0s 15ms/step - loss: 1.8530e-04
- mse: 1.8530e-04
Epoch 371/500
1/1 [=====] - 0s 13ms/step - loss: 1.8177e-04
- mse: 1.8177e-04
Epoch 372/500
1/1 [=====] - 0s 9ms/step - loss: 1.7830e-04
- mse: 1.7830e-04
Epoch 373/500
1/1 [=====] - 0s 8ms/step - loss: 1.7490e-04
- mse: 1.7490e-04
Epoch 374/500
1/1 [=====] - 0s 6ms/step - loss: 1.7156e-04
- mse: 1.7156e-04
Epoch 375/500
1/1 [=====] - 0s 13ms/step - loss: 1.6829e-04
- mse: 1.6829e-04
Epoch 376/500
1/1 [=====] - 0s 22ms/step - loss: 1.6508e-04
- mse: 1.6508e-04
Epoch 377/500
1/1 [=====] - 0s 8ms/step - loss: 1.6193e-04
- mse: 1.6193e-04
Epoch 378/500
1/1 [=====] - 0s 6ms/step - loss: 1.5884e-04
- mse: 1.5884e-04
Epoch 379/500
1/1 [=====] - 0s 8ms/step - loss: 1.5581e-04
- mse: 1.5581e-04
Epoch 380/500
1/1 [=====] - 0s 4ms/step - loss: 1.5284e-04
```

```
- mse: 1.5284e-04
Epoch 381/500
1/1 [=====] - 0s 4ms/step - loss: 1.4993e-04
- mse: 1.4993e-04
Epoch 382/500
1/1 [=====] - 0s 12ms/step - loss: 1.4706e-04
- mse: 1.4706e-04
Epoch 383/500
1/1 [=====] - 0s 9ms/step - loss: 1.4426e-04
- mse: 1.4426e-04
Epoch 384/500
1/1 [=====] - 0s 8ms/step - loss: 1.4151e-04
- mse: 1.4151e-04
Epoch 385/500
1/1 [=====] - 0s 6ms/step - loss: 1.3881e-04
- mse: 1.3881e-04
Epoch 386/500
1/1 [=====] - 0s 6ms/step - loss: 1.3616e-04
- mse: 1.3616e-04
Epoch 387/500
1/1 [=====] - 0s 8ms/step - loss: 1.3356e-04
- mse: 1.3356e-04
Epoch 388/500
1/1 [=====] - 0s 4ms/step - loss: 1.3102e-04
- mse: 1.3102e-04
Epoch 389/500
1/1 [=====] - 0s 9ms/step - loss: 1.2852e-04
- mse: 1.2852e-04
Epoch 390/500
1/1 [=====] - 0s 10ms/step - loss: 1.2607e-04
- mse: 1.2607e-04
Epoch 391/500
1/1 [=====] - 0s 8ms/step - loss: 1.2366e-04
- mse: 1.2366e-04
Epoch 392/500
1/1 [=====] - 0s 4ms/step - loss: 1.2130e-04
- mse: 1.2130e-04
Epoch 393/500
1/1 [=====] - 0s 12ms/step - loss: 1.1899e-04
- mse: 1.1899e-04
Epoch 394/500
1/1 [=====] - 0s 7ms/step - loss: 1.1672e-04
- mse: 1.1672e-04
Epoch 395/500
1/1 [=====] - 0s 6ms/step - loss: 1.1449e-04
- mse: 1.1449e-04
Epoch 396/500
1/1 [=====] - 0s 6ms/step - loss: 1.1231e-04
- mse: 1.1231e-04
```

```
Epoch 397/500
1/1 [=====] - 0s 6ms/step - loss: 1.1016e-04
- mse: 1.1016e-04
Epoch 398/500
1/1 [=====] - 0s 6ms/step - loss: 1.0807e-04
- mse: 1.0807e-04
Epoch 399/500
1/1 [=====] - 0s 14ms/step - loss: 1.0600e-04
- mse: 1.0600e-04
Epoch 400/500
1/1 [=====] - 0s 9ms/step - loss: 1.0398e-04
- mse: 1.0398e-04
Epoch 401/500
1/1 [=====] - 0s 7ms/step - loss: 1.0200e-04
- mse: 1.0200e-04
Epoch 402/500
1/1 [=====] - 0s 7ms/step - loss: 1.0005e-04
- mse: 1.0005e-04
Epoch 403/500
1/1 [=====] - 0s 9ms/step - loss: 9.8142e-05
- mse: 9.8142e-05
Epoch 404/500
1/1 [=====] - 0s 13ms/step - loss: 9.6271e-05
- mse: 9.6271e-05
Epoch 405/500
1/1 [=====] - 0s 6ms/step - loss: 9.4433e-05
- mse: 9.4433e-05
Epoch 406/500
1/1 [=====] - 0s 8ms/step - loss: 9.2632e-05
- mse: 9.2632e-05
Epoch 407/500
1/1 [=====] - 0s 33ms/step - loss: 9.0865e-05
- mse: 9.0865e-05
Epoch 408/500
1/1 [=====] - 0s 12ms/step - loss: 8.9132e-05
- mse: 8.9132e-05
Epoch 409/500
1/1 [=====] - 0s 9ms/step - loss: 8.7431e-05
- mse: 8.7431e-05
Epoch 410/500
1/1 [=====] - 0s 6ms/step - loss: 8.5761e-05
- mse: 8.5761e-05
Epoch 411/500
1/1 [=====] - 0s 15ms/step - loss: 8.4128e-05
- mse: 8.4128e-05
Epoch 412/500
1/1 [=====] - 0s 7ms/step - loss: 8.2522e-05
- mse: 8.2522e-05
Epoch 413/500
```

```
1/1 [=====] - 0s 7ms/step - loss: 8.0950e-05
- mse: 8.0950e-05
Epoch 414/500
1/1 [=====] - 0s 6ms/step - loss: 7.9403e-05
- mse: 7.9403e-05
Epoch 415/500
1/1 [=====] - 0s 9ms/step - loss: 7.7892e-05
- mse: 7.7892e-05
Epoch 416/500
1/1 [=====] - 0s 7ms/step - loss: 7.6404e-05
- mse: 7.6404e-05
Epoch 417/500
1/1 [=====] - 0s 7ms/step - loss: 7.4948e-05
- mse: 7.4948e-05
Epoch 418/500
1/1 [=====] - 0s 7ms/step - loss: 7.3519e-05
- mse: 7.3519e-05
Epoch 419/500
1/1 [=====] - 0s 7ms/step - loss: 7.2114e-05
- mse: 7.2114e-05
Epoch 420/500
1/1 [=====] - 0s 8ms/step - loss: 7.0738e-05
- mse: 7.0738e-05
Epoch 421/500
1/1 [=====] - 0s 9ms/step - loss: 6.9390e-05
- mse: 6.9390e-05
Epoch 422/500
1/1 [=====] - 0s 5ms/step - loss: 6.8068e-05
- mse: 6.8068e-05
Epoch 423/500
1/1 [=====] - 0s 7ms/step - loss: 6.6769e-05
- mse: 6.6769e-05
Epoch 424/500
1/1 [=====] - 0s 10ms/step - loss: 6.5493e-05
- mse: 6.5493e-05
Epoch 425/500
1/1 [=====] - 0s 5ms/step - loss: 6.4246e-05
- mse: 6.4246e-05
Epoch 426/500
1/1 [=====] - 0s 8ms/step - loss: 6.3018e-05
- mse: 6.3018e-05
Epoch 427/500
1/1 [=====] - 0s 8ms/step - loss: 6.1819e-05
- mse: 6.1819e-05
Epoch 428/500
1/1 [=====] - 0s 8ms/step - loss: 6.0638e-05
- mse: 6.0638e-05
Epoch 429/500
1/1 [=====] - 0s 19ms/step - loss: 5.9481e-05
```



```
- mse: 5.9481e-05
Epoch 430/500
1/1 [=====] - 0s 9ms/step - loss: 5.8347e-05
- mse: 5.8347e-05
Epoch 431/500
1/1 [=====] - 0s 9ms/step - loss: 5.7234e-05
- mse: 5.7234e-05
Epoch 432/500
1/1 [=====] - 0s 15ms/step - loss: 5.6141e-05
- mse: 5.6141e-05
Epoch 433/500
1/1 [=====] - 0s 6ms/step - loss: 5.5071e-05
- mse: 5.5071e-05
Epoch 434/500
1/1 [=====] - 0s 10ms/step - loss: 5.4021e-05
- mse: 5.4021e-05
Epoch 435/500
1/1 [=====] - 0s 4ms/step - loss: 5.2991e-05
- mse: 5.2991e-05
Epoch 436/500
1/1 [=====] - 0s 5ms/step - loss: 5.1979e-05
- mse: 5.1979e-05
Epoch 437/500
1/1 [=====] - 0s 7ms/step - loss: 5.0989e-05
- mse: 5.0989e-05
Epoch 438/500
1/1 [=====] - 0s 5ms/step - loss: 5.0018e-05
- mse: 5.0018e-05
Epoch 439/500
1/1 [=====] - 0s 8ms/step - loss: 4.9064e-05
- mse: 4.9064e-05
Epoch 440/500
1/1 [=====] - 0s 6ms/step - loss: 4.8127e-05
- mse: 4.8127e-05
Epoch 441/500
1/1 [=====] - 0s 8ms/step - loss: 4.7209e-05
- mse: 4.7209e-05
Epoch 442/500
1/1 [=====] - 0s 24ms/step - loss: 4.6309e-05
- mse: 4.6309e-05
Epoch 443/500
1/1 [=====] - 0s 5ms/step - loss: 4.5426e-05
- mse: 4.5426e-05
Epoch 444/500
1/1 [=====] - 0s 7ms/step - loss: 4.4558e-05
- mse: 4.4558e-05
Epoch 445/500
1/1 [=====] - 0s 6ms/step - loss: 4.3708e-05
- mse: 4.3708e-05
```

```
Epoch 446/500
1/1 [=====] - 0s 10ms/step - loss: 4.2875e-05
- mse: 4.2875e-05
Epoch 447/500
1/1 [=====] - 0s 7ms/step - loss: 4.2056e-05
- mse: 4.2056e-05
Epoch 448/500
1/1 [=====] - 0s 7ms/step - loss: 4.1255e-05
- mse: 4.1255e-05
Epoch 449/500
1/1 [=====] - 0s 16ms/step - loss: 4.0467e-05
- mse: 4.0467e-05
Epoch 450/500
1/1 [=====] - 0s 6ms/step - loss: 3.9696e-05
- mse: 3.9696e-05
Epoch 451/500
1/1 [=====] - 0s 20ms/step - loss: 3.8938e-05
- mse: 3.8938e-05
Epoch 452/500
1/1 [=====] - 0s 17ms/step - loss: 3.8195e-05
- mse: 3.8195e-05
Epoch 453/500
1/1 [=====] - 0s 12ms/step - loss: 3.7467e-05
- mse: 3.7467e-05
Epoch 454/500
1/1 [=====] - 0s 9ms/step - loss: 3.6752e-05
- mse: 3.6752e-05
Epoch 455/500
1/1 [=====] - 0s 6ms/step - loss: 3.6050e-05
- mse: 3.6050e-05
Epoch 456/500
1/1 [=====] - 0s 9ms/step - loss: 3.5364e-05
- mse: 3.5364e-05
Epoch 457/500
1/1 [=====] - 0s 5ms/step - loss: 3.4690e-05
- mse: 3.4690e-05
Epoch 458/500
1/1 [=====] - 0s 5ms/step - loss: 3.4027e-05
- mse: 3.4027e-05
Epoch 459/500
1/1 [=====] - 0s 7ms/step - loss: 3.3378e-05
- mse: 3.3378e-05
Epoch 460/500
1/1 [=====] - 0s 5ms/step - loss: 3.2743e-05
- mse: 3.2743e-05
Epoch 461/500
1/1 [=====] - 0s 8ms/step - loss: 3.2116e-05
- mse: 3.2116e-05
Epoch 462/500
```

```
1/1 [=====] - 0s 4ms/step - loss: 3.1504e-05
- mse: 3.1504e-05
Epoch 463/500
1/1 [=====] - 0s 5ms/step - loss: 3.0903e-05
- mse: 3.0903e-05
Epoch 464/500
1/1 [=====] - 0s 4ms/step - loss: 3.0313e-05
- mse: 3.0313e-05
Epoch 465/500
1/1 [=====] - 0s 19ms/step - loss: 2.9735e-05
- mse: 2.9735e-05
Epoch 466/500
1/1 [=====] - 0s 7ms/step - loss: 2.9168e-05
- mse: 2.9168e-05
Epoch 467/500
1/1 [=====] - 0s 5ms/step - loss: 2.8612e-05
- mse: 2.8612e-05
Epoch 468/500
1/1 [=====] - 0s 6ms/step - loss: 2.8067e-05
- mse: 2.8067e-05
Epoch 469/500
1/1 [=====] - 0s 7ms/step - loss: 2.7531e-05
- mse: 2.7531e-05
Epoch 470/500
1/1 [=====] - 0s 5ms/step - loss: 2.7006e-05
- mse: 2.7006e-05
Epoch 471/500
1/1 [=====] - 0s 15ms/step - loss: 2.6490e-05
- mse: 2.6490e-05
Epoch 472/500
1/1 [=====] - 0s 8ms/step - loss: 2.5985e-05
- mse: 2.5985e-05
Epoch 473/500
1/1 [=====] - 0s 6ms/step - loss: 2.5489e-05
- mse: 2.5489e-05
Epoch 474/500
1/1 [=====] - 0s 8ms/step - loss: 2.5003e-05
- mse: 2.5003e-05
Epoch 475/500
1/1 [=====] - 0s 6ms/step - loss: 2.4525e-05
- mse: 2.4525e-05
Epoch 476/500
1/1 [=====] - 0s 5ms/step - loss: 2.4058e-05
- mse: 2.4058e-05
Epoch 477/500
1/1 [=====] - 0s 6ms/step - loss: 2.3599e-05
- mse: 2.3599e-05
Epoch 478/500
1/1 [=====] - 0s 5ms/step - loss: 2.3148e-05
```

```
- mse: 2.3148e-05
Epoch 479/500
1/1 [=====] - 0s 19ms/step - loss: 2.2707e-05
- mse: 2.2707e-05
Epoch 480/500
1/1 [=====] - 0s 41ms/step - loss: 2.2274e-05
- mse: 2.2274e-05
Epoch 481/500
1/1 [=====] - 0s 13ms/step - loss: 2.1850e-05
- mse: 2.1850e-05
Epoch 482/500
1/1 [=====] - 0s 48ms/step - loss: 2.1433e-05
- mse: 2.1433e-05
Epoch 483/500
1/1 [=====] - 0s 35ms/step - loss: 2.1024e-05
- mse: 2.1024e-05
Epoch 484/500
1/1 [=====] - 0s 12ms/step - loss: 2.0623e-05
- mse: 2.0623e-05
Epoch 485/500
1/1 [=====] - 0s 11ms/step - loss: 2.0229e-05
- mse: 2.0229e-05
Epoch 486/500
1/1 [=====] - 0s 9ms/step - loss: 1.9843e-05
- mse: 1.9843e-05
Epoch 487/500
1/1 [=====] - 0s 13ms/step - loss: 1.9464e-05
- mse: 1.9464e-05
Epoch 488/500
1/1 [=====] - 0s 15ms/step - loss: 1.9094e-05
- mse: 1.9094e-05
Epoch 489/500
1/1 [=====] - 0s 22ms/step - loss: 1.8730e-05
- mse: 1.8730e-05
Epoch 490/500
1/1 [=====] - 0s 6ms/step - loss: 1.8372e-05
- mse: 1.8372e-05
Epoch 491/500
1/1 [=====] - 0s 7ms/step - loss: 1.8022e-05
- mse: 1.8022e-05
Epoch 492/500
1/1 [=====] - 0s 13ms/step - loss: 1.7678e-05
- mse: 1.7678e-05
Epoch 493/500
1/1 [=====] - 0s 28ms/step - loss: 1.7342e-05
- mse: 1.7342e-05
Epoch 494/500
1/1 [=====] - 0s 30ms/step - loss: 1.7011e-05
- mse: 1.7011e-05
```

```
Epoch 495/500
1/1 [=====] - 0s 6ms/step - loss: 1.6685e-05
- mse: 1.6685e-05
Epoch 496/500
1/1 [=====] - 0s 11ms/step - loss: 1.6367e-05
- mse: 1.6367e-05
Epoch 497/500
1/1 [=====] - 0s 5ms/step - loss: 1.6055e-05
- mse: 1.6055e-05
Epoch 498/500
1/1 [=====] - 0s 7ms/step - loss: 1.5749e-05
- mse: 1.5749e-05
Epoch 499/500
1/1 [=====] - 0s 3ms/step - loss: 1.5448e-05
- mse: 1.5448e-05
Epoch 500/500
1/1 [=====] - 0s 15ms/step - loss: 1.5154e-05
- mse: 1.5154e-05
```

As you can see, the loss went from $1.2677e-05$ all the way down to $8.5635e-10$. If you run it again, these values may change. There are so many randomness involved in neural network training. Take an example, weights initialization is random.

2.5 Evaluating a Model

After we have trained the model, the next step is to evaluate it.

But first off, we can plot the loss versus the epochs to see how it performed. Plotting the model metrics is a fundamental step in performing the error analysis.

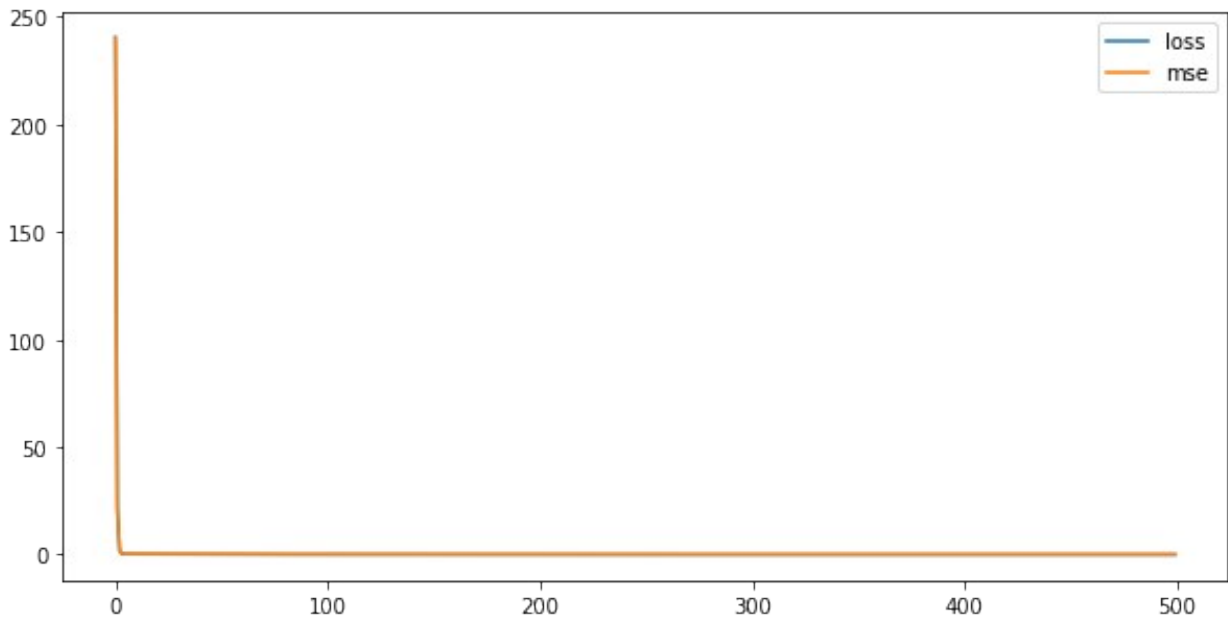
The training metric `mse` and `loss` are contained in `history.history` and the number of epochs are in `history.epoch`.

```
loss_df = pd.DataFrame(history.history)

# Plot loss vs epochs

loss_df.plot(figsize=(10,5))

<matplotlib.axes._subplots.AxesSubplot at 0x7f8c95ef28d0>
```



There are couple of things to note from the above graph.

- First off, the loss and metric we are tracking are all similar. They are both `mean_squared_error` or `mse`.
- The model had no improvement from 80(approx) epochs. This means that 500 epochs was too much, and instead of burning resources or compute power, we would have trained for fewer epochs since the model does not show a significant improvements in the later epochs.

Training for many epochs beyond what's needed is usually the cause of `overfitting`. We will learn more about overfitting later, but simply, it's when the model is so good on the training set but very poor on the test set or the new data. So, in our case we are forcing the model to fit the data by training it too much. Overfitting can also be caused by using bigger models (for small dataset), etc...

That said, let's make some predictions on unseen data. For simplicity, let's predict `y` of `X=30`. Remember that our equation is $y=2X+1$, so with $x=30$, y should be equal to 61.

```
model.predict([30.0])  
array([[61.014885]], dtype=float32)
```

Wow! That's so impressive.

The model was able to determine the relationship between `X` and `y` and can use that relationship to predict `y` for unseen values of `X`.

One thing to note that it is not guaranteed to get the exact predictions, say 61. This is because there are so many randomness and probabilities involved behind the scene.

One last thing we can try is to get the model parameters, that is weight and bias. And their values should be close to the coefficient and intercept of our linear equation, $y=2X+1$. 2 is coefficient(or weight), and 1 is the intercept(or bias).

```
# Getting the model weights
```

```
model.get_weights()
```

```
[array([[2.0006804]], dtype=float32), array([0.994472],  
dtype=float32)]
```

So, as you can see, the model learned that the relationship between X and y is $y=2.0008771 + 0.9928744$ and this is very close to $y=2X+1$. Something intriguing here is that there is no where told the model such relationship - it simply learned it observing the data that we provided, and this is the basis of the idea that machine learning is used to extract patterns in data.

With complex data, you might not get such intuition because there will be so many parameters, but in our case, the model was simple, the data was simple, and that allowed us to uncover the principal idea behind machine learning - learning the relationship/pattern between the input features and labels, and using such relationship to make predictions on unseen data.

2.6 Improving the results

Ideally, your model will likely not be good at the first.

You will need to tweak some hyperparameters, or even improve the data. Also, there is a notion that [machine learning model is only 5% of what are to be done to ship a working machine learning system](#). So, often, all you need is to improve the data than improving the model. There are even state of the art and open source models that you can take an advantages of if you have good data. And in those intances, building model will be out of the equation.

But of course, improving the model and performing error analysis is not a trivial task and will depend on the results of the model on training and testing set. Here is some ideas that can guide you:

- If the model is not doing well on the training data, it's a clue that the input data (X) doesn't contain the useful information needed to predict the output y. Or put it simply, the input features do not have high predictive power. The right thing to do here is to improve the data. Otherwise, the problem will perssit.
- If the model is doing well on the training data but poorly on the testing data, it maybe that you overfitted the training data and that resulted in model failing to generalize on test/new data. Overfitting is one thing, there maybe other things not going well or worth improving. The right thing to do here is to plot the learning curve and see what's to be done based off what you are seeing.

Up to now, we have come a long way doing regression with neural networks. We have learned how to create a simple data, how to create, train, and compile a simple model, evaluating the results, and we saw some ideas on performing error analysis.

We started simple with the goal of getting prepared to take a step further into real world scenarios. I wanted to jump quickly to bigger models and computer vision things but I remembered that quite often, it is understanding the basics that can set us off for understanding the bigger picture.

To make it more exciting, let's not stop on linear equation (we could after all, we did regression already), but let's step into real world dataset, still practicing regression.

3. Going Beyond: A Real world dataset

Welcome to the second part of the notebook, where we leap into real world scenarios.

Still doing regression, we will use the real world forest dataset to predict the burned area of forest fires, in the northeast region of Portugal, by using meteorological and other data.

You can learn more about the dataset [here](#).

3.1 Loading the data

Before loading the dataset, I will first import all relevant imports.

Here are the information about the attributes:

1. X - x-axis spatial coordinate within the Montesinho park map: 1 to 9
2. Y - y-axis spatial coordinate within the Montesinho park map: 2 to 9
3. month - month of the year: 'jan' to 'dec'
4. day - day of the week: 'mon' to 'sun'
5. FFMCI - FFMCI index from the FWI system: 18.7 to 96.20 (đánh giá độ ẩm của các loại nhiên liệu mịn trên mặt đất rừng như lá khô, cỏ)
6. DMC - DMC index from the FWI system: 1.1 to 291.3 (đo lường độ ẩm của các lớp hữu cơ đệm dưới bề mặt.)
7. DC - DC index from the FWI system: 7.9 to 860.6 (Đánh giá ảnh hưởng của hạn hán dài hạn đến độ ẩm của các lớp hữu cơ sâu hơn và gỗ lớn.)
8. ISI - ISI index from the FWI system: 0.0 to 56.10 (chỉ số kết hợp giữa tốc độ gió và FFMCI. Nó đánh giá khả năng lan truyền của đám cháy mà không tính đến ảnh hưởng của nhiên liệu có kích thước lớn hơn)
9. temp - temperature in Celsius degrees: 2.2 to 33.30
10. RH - relative humidity in %: 15.0 to 100
11. wind - wind speed in km/h: 0.40 to 9.40
12. rain - outside rain in mm/m2 : 0.0 to 6.4
13. area - the burned area of the forest (in ha): 0.00 to 1090.84

Source: https://archive.ics.uci.edu/ml/datasets/Forest+**Fires

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Let's download the dataset and load it into a Pandas dataframe using `pd.read_csv()`,

```
dataset_url = 'https://archive.ics.uci.edu/ml/machine-learning-
databases/forest-fires/forestfires.csv'
```

```
forest_df = pd.read_csv(dataset_url)
```

Let's see the features and their data types

```
forest_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 517 entries, 0 to 516
Data columns (total 13 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   X            517 non-null    int64
 1   Y            517 non-null    int64
 2   month        517 non-null    object
 3   day          517 non-null    object
 4   FFMC         517 non-null    float64
 5   DMC          517 non-null    float64
 6   DC           517 non-null    float64
 7   ISI          517 non-null    float64
 8   temp         517 non-null    float64
 9   RH           517 non-null    int64
10   wind         517 non-null    float64
11   rain         517 non-null    float64
12   area         517 non-null    float64
dtypes: float64(8), int64(3), object(2)
memory usage: 52.6+ KB
```

The dataset contains 517 examples and 13 columns, 12 features and 1 label (`areas`).

```
print(forest_df.shape)

(517, 13)
```

3.2 Looking in the data

We will not go deep into analysis, but let's try to learn about the data we have. Before that, I will first split the dataset into training and test set.

I will use Scikit-Learn `train_test_split`.

```
from sklearn.model_selection import train_test_split

train_data, test_data = train_test_split(forest_df, test_size=0.3,
random_state=42)

print('The shape of training data: {} \n The shape of testing data: {}'
.format(train_data.shape, test_data.shape))

The shape of training data: (361, 13)
The shape of testing data: (156, 13)
```

Let's peep into the data.

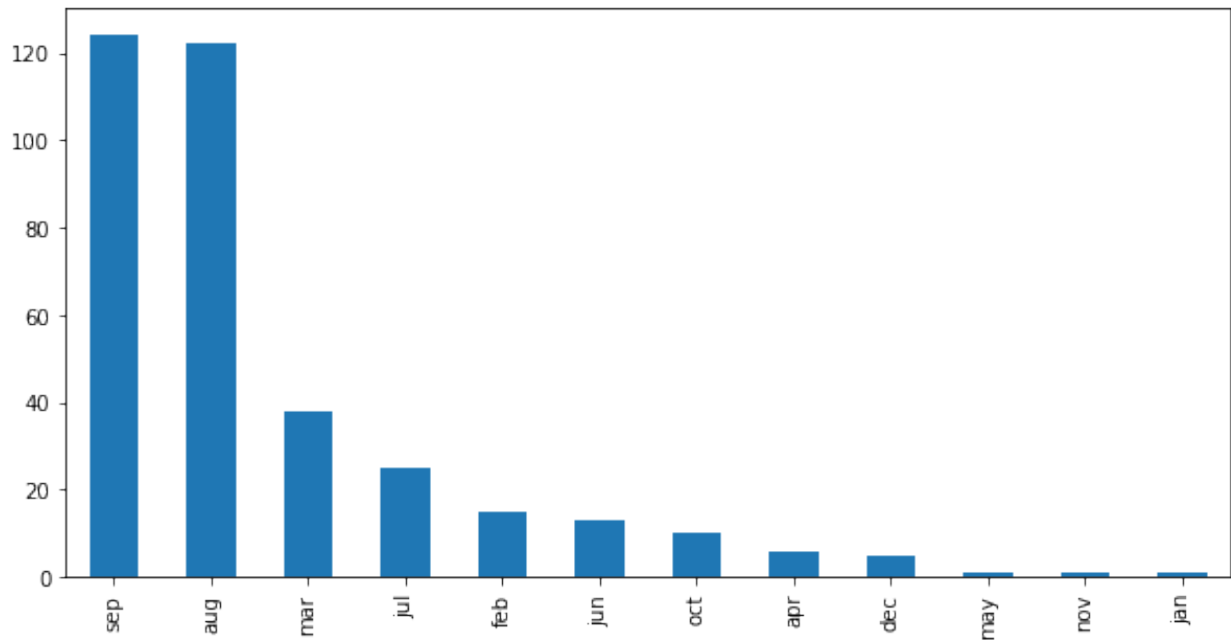
```
train_data.head(5)
```

	X	Y	month	day	FFMC	DMC	DC	ISI	temp	RH	wind	rain
area												
311	6	3	sep	sun	92.4	105.8	758.1	9.9	24.8	28	1.8	0.0
14.29												
368	6	5	sep	sat	91.2	94.3	744.4	8.4	16.8	47	4.9	0.0
12.64												
23	7	4	aug	sat	90.2	110.9	537.4	6.2	19.5	43	5.8	0.0
0.00												
271	8	6	aug	tue	92.1	152.6	658.2	14.3	20.1	58	4.5	0.0
9.27												
299	6	5	jun	sat	53.4	71.0	233.8	0.4	10.6	90	2.7	0.0
0.00												

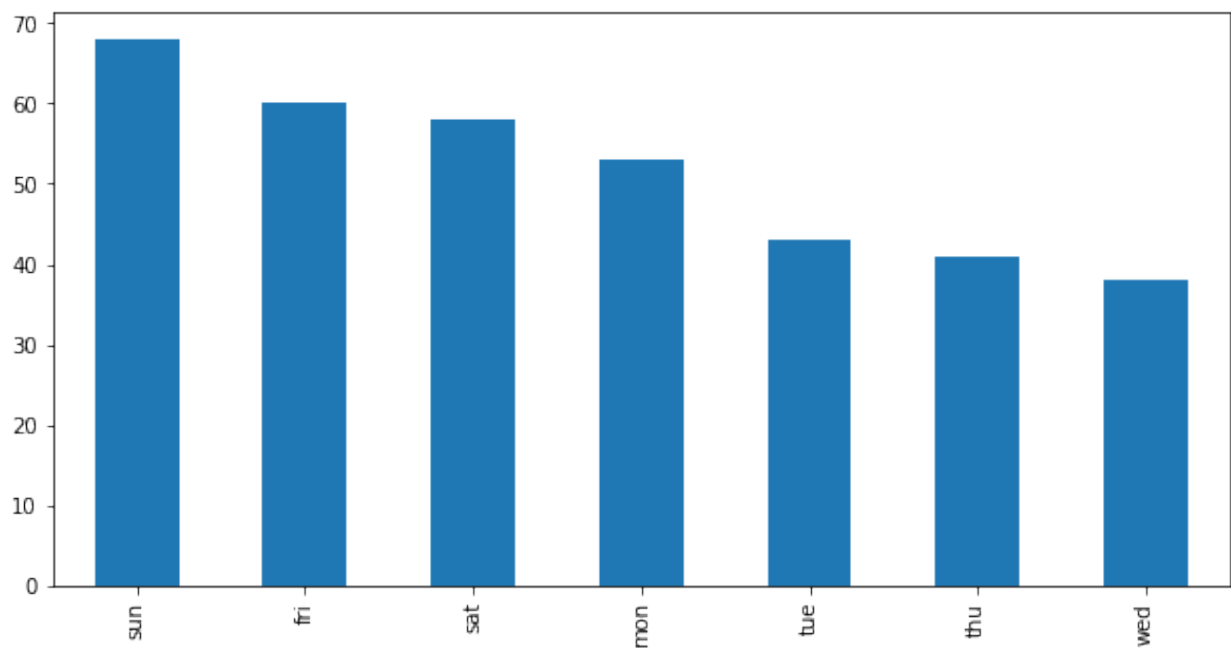
It seems that we have two categorical features, `month` and `day`. We will remember to encode them. For now we can see the number of samples in each month and later in each day.

```
train_data['month'].value_counts().plot(kind='bar', figsize=(10,5))

<matplotlib.axes._subplots.AxesSubplot at 0x7f8c8ef60550>
```



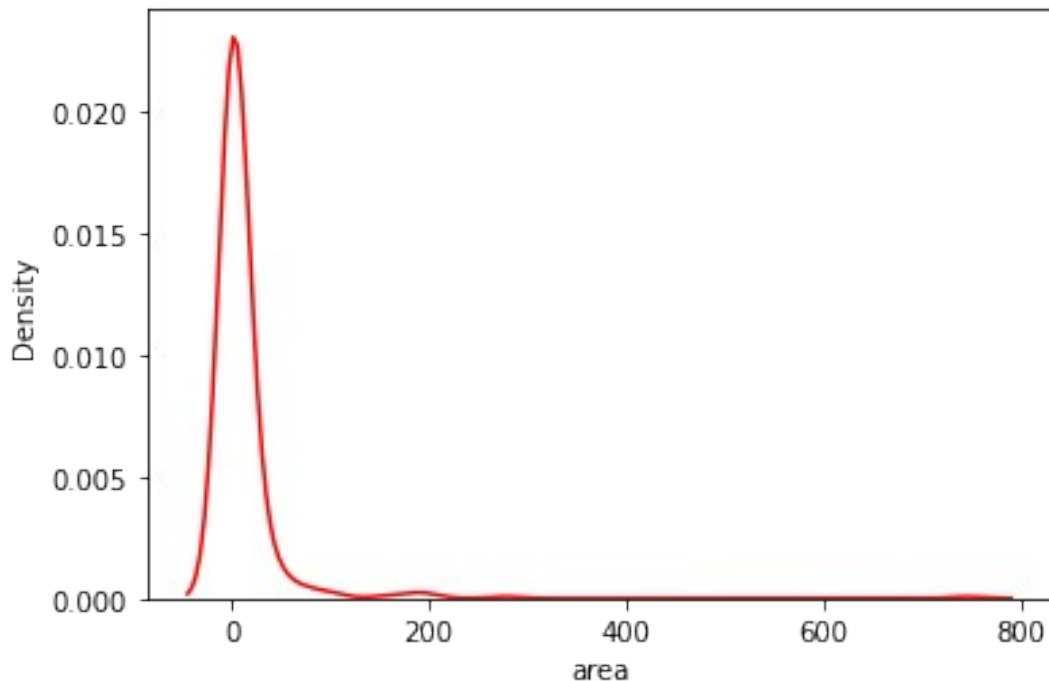
```
train_data['day'].value_counts().plot(kind='bar', figsize=(10,5))  
<matplotlib.axes._subplots.AxesSubplot at 0x7f8c8d490f90>
```



We can also check the distribution of the area. Area is very skewed, you can see that most values are very close to zero.

```
sns.kdeplot(data=train_data, x='area', color='red')
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f8c8b079390>
```



3.3 Preparing the Data for the Model

Here we will do two things, one is to normalize numerical features and the second is to encode categorical features. We can set up a pipeline to handle that.

For simplicity, we will use Scikit-Learn processing functions.

I will first separate features and label. We can use a function that can also be applied to test set.

```
def get_feats_and_labels(data, label):  
    """ Take data and label as inputs, return features and labels  
    separated """  
  
    data_feats = data.drop(label, axis=1)  
    data_label = data[label]  
  
    return data_feats, data_label
```

Let's use the function created above to get the features and labels.

```
train_feats, train_label = get_feats_and_labels(train_data, 'area')  
  
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
```

```

scaler = StandardScaler()
encoder = OrdinalEncoder()

# The column transformer requires lists of features
num_feats = ['X', 'Y', 'FFMC', 'DMC', 'DC', 'ISI', 'temp', 'RH',
             'wind', 'rain']
cat_feats = ['month', 'day']

# define the pipeline to scale the numeric features and handle
categorical features
final_pipe = ColumnTransformer([
    ('num', scaler, num_feats),
    ('cat', encoder, cat_feats)
])

training_data_prepared = final_pipe.fit_transform(train_feats)

```

Now, we can see the shape of the transformed dataset. It is a NumPy array.

```

training_data_prepared.shape
(361, 12)
type(training_data_prepared)
numpy.ndarray

```

Also let's transform the test set. Note that for the test set, we don't `fit_transform()`.

I will get the features and labels separated first.

```

test_feats, test_label = get_feats_and_labels(test_data, 'area')

```

And now we transform the test features.

```

test_data_prepared = final_pipe.transform(test_feats)

```

Let's convert train and test labels to NumPy array.

```

train_label = train_label.to_numpy()
test_label = test_label.to_numpy()

```

3.4 Creating, Compiling and Training a Model

Now that our data is prepared, it's time to create a neural network regressor.

Like in the first example, we will use the [Sequential API](#).

Everytime we are creating a model in TensorFlow, we have to specify the input shape. In this example, the input shape will be:

```
input_shape = training_data_prepared.shape[1:]
input_shape

(12,)

model = keras.models.Sequential([

    # The first layers must specify the input shape always
    keras.layers.Dense(12, activation='relu',
input_shape=input_shape),
    keras.layers.Dense(24, activation='relu'),

    # The last layer usually doesn't have activation function
in regression
    keras.layers.Dense(1)

])

# Now we compile the model
model.compile(loss='mean_squared_error', optimizer='adam')
```

Let's see the model summary.

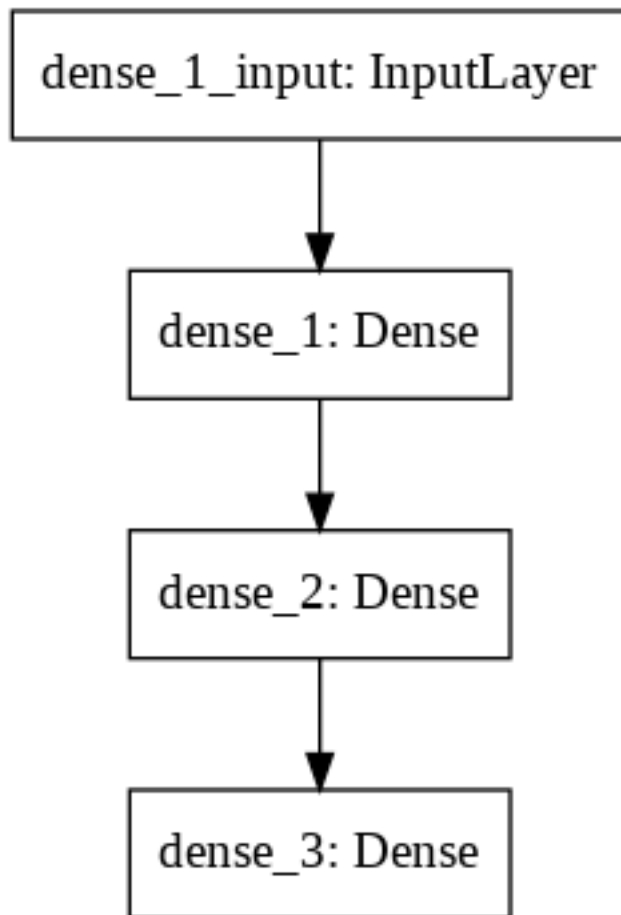
```
model.summary()
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 12)	156
dense_2 (Dense)	(None, 24)	312
dense_3 (Dense)	(None, 1)	25

```
Total params: 493
Trainable params: 493
Non-trainable params: 0
```

Remember, when the model is created in TensorFlow, it is like an empty graphs. We can even visualize it.

```
from tensorflow.keras.utils import plot_model  
plot_model(model, to_file='model.png')
```



Before training or fitting the model to the data, a model is nothing other than empty graphs.

Now let's train the model.

```
history = model.fit(training_data_prepared, train_label,  
                    validation_data = (test_data_prepared,  
test_label), epochs=50)  
  
Epoch 1/50  
12/12 [=====] - 1s 17ms/step - loss:  
2500.8550 - val_loss: 8183.7715  
Epoch 2/50  
12/12 [=====] - 0s 3ms/step - loss: 2489.0750  
- val_loss: 8166.7290  
Epoch 3/50
```

```
12/12 [=====] - 0s 4ms/step - loss: 2480.3232
- val_loss: 8150.3276
Epoch 4/50
12/12 [=====] - 0s 3ms/step - loss: 2470.4707
- val_loss: 8137.5972
Epoch 5/50
12/12 [=====] - 0s 4ms/step - loss: 2462.4663
- val_loss: 8124.1401
Epoch 6/50
12/12 [=====] - 0s 5ms/step - loss: 2453.9116
- val_loss: 8110.1162
Epoch 7/50
12/12 [=====] - 0s 3ms/step - loss: 2445.4663
- val_loss: 8095.4053
Epoch 8/50
12/12 [=====] - 0s 6ms/step - loss: 2435.2563
- val_loss: 8077.7803
Epoch 9/50
12/12 [=====] - 0s 3ms/step - loss: 2426.2720
- val_loss: 8056.7891
Epoch 10/50
12/12 [=====] - 0s 5ms/step - loss: 2415.6418
- val_loss: 8038.3853
Epoch 11/50
12/12 [=====] - 0s 4ms/step - loss: 2405.9746
- val_loss: 8021.0791
Epoch 12/50
12/12 [=====] - 0s 5ms/step - loss: 2399.9617
- val_loss: 8000.2622
Epoch 13/50
12/12 [=====] - 0s 5ms/step - loss: 2388.9194
- val_loss: 7986.3862
Epoch 14/50
12/12 [=====] - 0s 3ms/step - loss: 2383.2515
- val_loss: 7973.9990
Epoch 15/50
12/12 [=====] - 0s 3ms/step - loss: 2377.0486
- val_loss: 7962.8188
Epoch 16/50
12/12 [=====] - 0s 5ms/step - loss: 2372.3306
- val_loss: 7954.1953
Epoch 17/50
12/12 [=====] - 0s 4ms/step - loss: 2369.2825
- val_loss: 7946.6025
Epoch 18/50
12/12 [=====] - 0s 5ms/step - loss: 2364.2703
- val_loss: 7942.8389
Epoch 19/50
12/12 [=====] - 0s 4ms/step - loss: 2361.3081
```



```
- val_loss: 7942.8628
Epoch 20/50
12/12 [=====] - 0s 5ms/step - loss: 2358.3665
- val_loss: 7940.8789
Epoch 21/50
12/12 [=====] - 0s 5ms/step - loss: 2354.2932
- val_loss: 7937.1777
Epoch 22/50
12/12 [=====] - 0s 4ms/step - loss: 2351.3564
- val_loss: 7934.7852
Epoch 23/50
12/12 [=====] - 0s 5ms/step - loss: 2349.0132
- val_loss: 7931.3711
Epoch 24/50
12/12 [=====] - 0s 5ms/step - loss: 2345.8608
- val_loss: 7930.4922
Epoch 25/50
12/12 [=====] - 0s 5ms/step - loss: 2343.2444
- val_loss: 7931.0679
Epoch 26/50
12/12 [=====] - 0s 3ms/step - loss: 2339.5933
- val_loss: 7927.4663
Epoch 27/50
12/12 [=====] - 0s 5ms/step - loss: 2337.6555
- val_loss: 7924.7412
Epoch 28/50
12/12 [=====] - 0s 3ms/step - loss: 2334.4104
- val_loss: 7924.7734
Epoch 29/50
12/12 [=====] - 0s 4ms/step - loss: 2332.6509
- val_loss: 7925.0747
Epoch 30/50
12/12 [=====] - 0s 4ms/step - loss: 2328.8726
- val_loss: 7924.2202
Epoch 31/50
12/12 [=====] - 0s 4ms/step - loss: 2327.3474
- val_loss: 7927.2397
Epoch 32/50
12/12 [=====] - 0s 4ms/step - loss: 2323.9705
- val_loss: 7927.1152
Epoch 33/50
12/12 [=====] - 0s 5ms/step - loss: 2321.2361
- val_loss: 7925.8413
Epoch 34/50
12/12 [=====] - 0s 4ms/step - loss: 2318.6199
- val_loss: 7926.0889
Epoch 35/50
12/12 [=====] - 0s 5ms/step - loss: 2317.1487
- val_loss: 7927.2734
```

```
Epoch 36/50
12/12 [=====] - 0s 4ms/step - loss: 2314.9399
- val_loss: 7932.5273
Epoch 37/50
12/12 [=====] - 0s 5ms/step - loss: 2310.1355
- val_loss: 7924.5747
Epoch 38/50
12/12 [=====] - 0s 3ms/step - loss: 2305.5168
- val_loss: 7923.9902
Epoch 39/50
12/12 [=====] - 0s 5ms/step - loss: 2304.0037
- val_loss: 7927.2749
Epoch 40/50
12/12 [=====] - 0s 5ms/step - loss: 2300.7092
- val_loss: 7922.8140
Epoch 41/50
12/12 [=====] - 0s 4ms/step - loss: 2298.9998
- val_loss: 7923.1523
Epoch 42/50
12/12 [=====] - 0s 4ms/step - loss: 2296.6914
- val_loss: 7924.2998
Epoch 43/50
12/12 [=====] - 0s 4ms/step - loss: 2294.8440
- val_loss: 7925.5903
Epoch 44/50
12/12 [=====] - 0s 4ms/step - loss: 2292.0125
- val_loss: 7931.0190
Epoch 45/50
12/12 [=====] - 0s 6ms/step - loss: 2290.0403
- val_loss: 7931.6738
Epoch 46/50
12/12 [=====] - 0s 5ms/step - loss: 2287.5166
- val_loss: 7933.3862
Epoch 47/50
12/12 [=====] - 0s 5ms/step - loss: 2285.0198
- val_loss: 7933.2266
Epoch 48/50
12/12 [=====] - 0s 5ms/step - loss: 2282.4080
- val_loss: 7934.9614
Epoch 49/50
12/12 [=====] - 0s 5ms/step - loss: 2281.5061
- val_loss: 7933.1396
Epoch 50/50
12/12 [=====] - 0s 4ms/step - loss: 2276.9304
- val_loss: 7934.7148
```

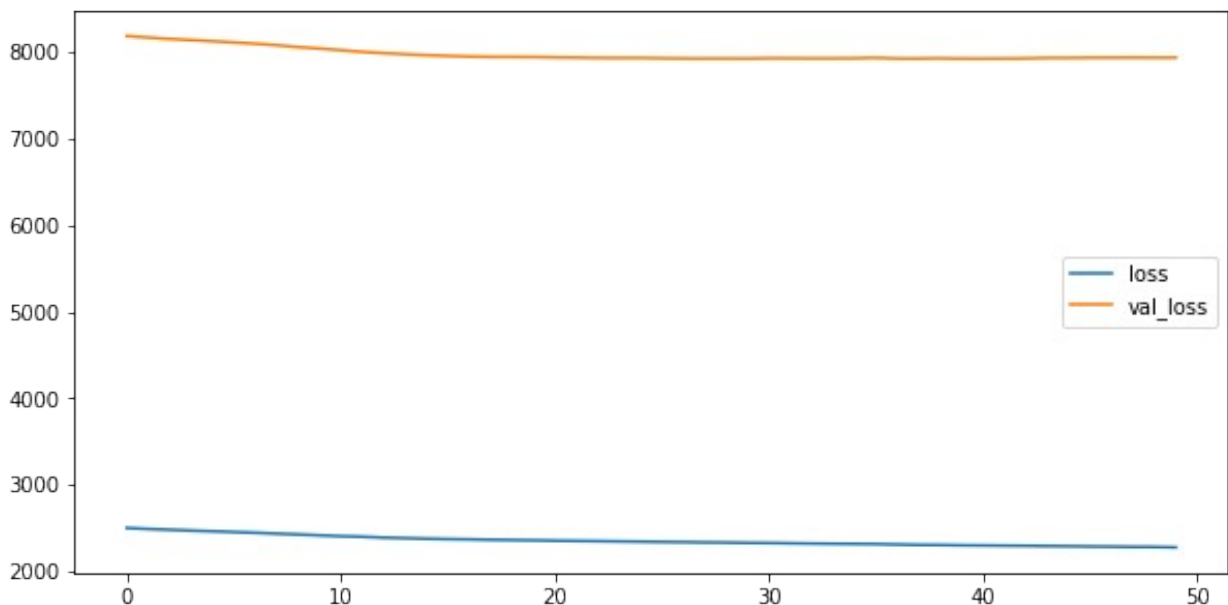
3.5 Evaluating a Model

After we have trained the model, the next step is to evaluate it.

But first off, we can plot the loss versus the epochs to see how it performed. Plotting the model metrics is a fundamental step in performing the error analysis.

`loss` and `val_loss` are contained in `history.history` and the number of epochs are in `history.epoch`.

```
loss_df = pd.DataFrame(history.history)
# Plot loss vs epochs
loss_df.plot(figsize=(10,5))
<matplotlib.axes._subplots.AxesSubplot at 0x7f8c8b06fe50>
```



```
model.evaluate(test_data_prepared, test_label)
5/5 [=====] - 0s 2ms/step - loss: 7934.7148
7934.71484375
```

The results are not impressive. Let's try to think why the model is not doing well. There are some few things to draw from the graph above.

- Ideally, both validation and training loss should decrease during training. If training doesn't decrease, it's very likely that the input features don't contain enough information to predict the output.

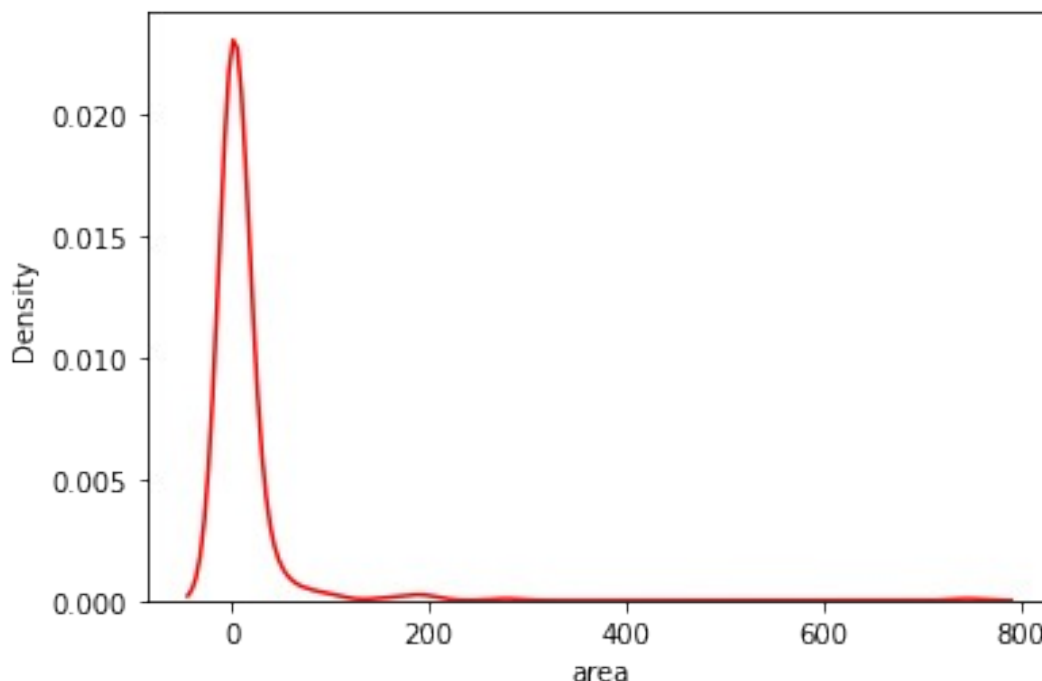
- loss là giá trị hàm mất mát (loss function) trên tập dữ liệu huấn luyện. val_loss là giá trị hàm mất mát trên tập dữ liệu kiểm chứng (validation set)
- Both `loss` and `val_loss` didn't improve alot, and there is no evidence that training for more epochs will improve the results. Quite the opposite, there is evidence that it will not improve.
- How about adding more layers, or neurons? There is a notion that a model is as good as the data it was trained on. In most cases, the sure thing to improve or add more data.

Let's see what we can improve.

3.6 Improving the Model

This data is very skewed. The burned area of the forest varies from 0.00 to 1090.84 but it's skewed to ward 0. Take a look at it again below...

```
sns.kdeplot(data=train_data, x='area', color='red')
<matplotlib.axes._subplots.AxesSubplot at 0x7f8ce932f5d0>
```



As the data source suggested, it may make sense to model it with logarithmic loss. Let's use `LogCosh` class that is available in [Keras Regression losses](#). That type of loss function computes the logarithm of the hyperbolic cosine of the prediction error.

Also, as the most values of the target label `area` falls between 0 and 1, we can use the `sigmoid` activation function so that the output of the network doesn't swing above such range.

1. Mean Squared Error (MSE): Khi sử dụng: Khi dữ liệu có phân phối chuẩn (normal distribution). Trong các bài toán hồi quy tuyến tính đơn giản. Khi outliers (giá trị bất thường) không phải là vấn đề lớn trong dữ liệu của bạn.

Ưu điểm: Dễ hiểu và diễn giải. Có đạo hàm đơn giản, thuận lợi cho việc tối ưu hóa. Hiệu quả trong việc tìm giá trị trung bình của dữ liệu.

Nhược điểm: Nhạy cảm với outliers, có thể bị ảnh hưởng nhiều bởi các giá trị cực đoan. Có thể gặp vấn đề khi xử lý dữ liệu có phạm vi giá trị rộng.

1. LogCosh: Khi sử dụng: Khi dữ liệu có phân phối lệch (skewed distribution). Khi bạn muốn xử lý outliers tốt hơn mà không loại bỏ chúng hoàn toàn. Trong các bài toán có phạm vi giá trị rộng, như dự đoán diện tích cháy rừng.

Ưu điểm: Ít nhạy cảm với outliers hơn MSE. Có đạo hàm mượt mà, giúp quá trình tối ưu hóa ổn định. Kết hợp ưu điểm của MSE (đối với các lỗi nhỏ) và MAE (đối với các lỗi lớn).

Nhược điểm: Khó diễn giải trực quan hơn so với MSE. Có thể không hiệu quả bằng MSE khi dữ liệu có phân phối chuẩn.

1. So sánh và lựa chọn: Đặc điểm dữ liệu: Nếu dữ liệu có phân phối chuẩn và ít outliers: Sử dụng MSE. Nếu dữ liệu có phân phối lệch hoặc có nhiều outliers: Cân nhắc LogCosh.

Mục tiêu của mô hình: Nếu bạn muốn mô hình tập trung vào xu hướng chung và ít bị ảnh hưởng bởi các điểm dữ liệu cực đoan: LogCosh có thể là lựa chọn tốt. Nếu bạn muốn mô hình nhạy cảm với mọi điểm dữ liệu, kể cả outliers: MSE có thể phù hợp hơn.

Thử nghiệm thực tế: Trong nhiều trường hợp, cách tốt nhất là thử nghiệm cả hai và so sánh kết quả. Đánh giá hiệu suất mô hình trên tập validation để xem hàm mất mát nào cho kết quả tốt hơn.

Trong trường hợp dữ liệu diện tích cháy rừng của bạn, với phạm vi giá trị rộng (0.00 đến 1090.84), LogCosh có thể là lựa chọn tốt vì nó xử lý tốt hơn với phạm vi giá trị lớn và có thể có outliers. Tuy nhiên, việc thử nghiệm cả hai phương pháp và so sánh kết quả vẫn là cách tiếp cận tốt nhất.

```
model_2 = keras.models.Sequential([  
    # The first layers must specify the input shape always  
    keras.layers.Dense(12, activation='relu',  
input_shape=input_shape),  
    keras.layers.Dense(6, activation='relu'),  
    # The last layer usually doesn't have activation function  
in regression  
    keras.layers.Dense(1, activation='sigmoid')  
])
```

```
# Now we compile the model
model_2.compile(loss='log_cosh', optimizer='adam')

history = model_2.fit(training_data_prepared, train_label,
                      validation_data = (test_data_prepared,
test_label), epochs=50)

Epoch 1/50
12/12 [=====] - 1s 13ms/step - loss: 11.5257 -
- val_loss: 14.5699
Epoch 2/50
12/12 [=====] - 0s 3ms/step - loss: 11.5059 -
val_loss: 14.5482
Epoch 3/50
12/12 [=====] - 0s 4ms/step - loss: 11.4805 -
val_loss: 14.5220
Epoch 4/50
12/12 [=====] - 0s 5ms/step - loss: 11.4523 -
val_loss: 14.4939
Epoch 5/50
12/12 [=====] - 0s 5ms/step - loss: 11.4251 -
val_loss: 14.4703
Epoch 6/50
12/12 [=====] - 0s 5ms/step - loss: 11.4017 -
val_loss: 14.4518
Epoch 7/50
12/12 [=====] - 0s 5ms/step - loss: 11.3838 -
val_loss: 14.4395
Epoch 8/50
12/12 [=====] - 0s 3ms/step - loss: 11.3710 -
val_loss: 14.4299
Epoch 9/50
12/12 [=====] - 0s 5ms/step - loss: 11.3635 -
val_loss: 14.4229
Epoch 10/50
12/12 [=====] - 0s 5ms/step - loss: 11.3571 -
val_loss: 14.4192
Epoch 11/50
12/12 [=====] - 0s 3ms/step - loss: 11.3530 -
val_loss: 14.4164
Epoch 12/50
12/12 [=====] - 0s 5ms/step - loss: 11.3500 -
val_loss: 14.4147
Epoch 13/50
12/12 [=====] - 0s 3ms/step - loss: 11.3472 -
val_loss: 14.4145
Epoch 14/50
12/12 [=====] - 0s 3ms/step - loss: 11.3447 -
val_loss: 14.4134
```

```
Epoch 15/50
12/12 [=====] - 0s 3ms/step - loss: 11.3421 -
val_loss: 14.4125
Epoch 16/50
12/12 [=====] - 0s 5ms/step - loss: 11.3404 -
val_loss: 14.4112
Epoch 17/50
12/12 [=====] - 0s 5ms/step - loss: 11.3383 -
val_loss: 14.4111
Epoch 18/50
12/12 [=====] - 0s 3ms/step - loss: 11.3368 -
val_loss: 14.4110
Epoch 19/50
12/12 [=====] - 0s 5ms/step - loss: 11.3355 -
val_loss: 14.4111
Epoch 20/50
12/12 [=====] - 0s 4ms/step - loss: 11.3342 -
val_loss: 14.4114
Epoch 21/50
12/12 [=====] - 0s 5ms/step - loss: 11.3332 -
val_loss: 14.4112
Epoch 22/50
12/12 [=====] - 0s 4ms/step - loss: 11.3320 -
val_loss: 14.4116
Epoch 23/50
12/12 [=====] - 0s 4ms/step - loss: 11.3310 -
val_loss: 14.4113
Epoch 24/50
12/12 [=====] - 0s 5ms/step - loss: 11.3302 -
val_loss: 14.4119
Epoch 25/50
12/12 [=====] - 0s 5ms/step - loss: 11.3293 -
val_loss: 14.4125
Epoch 26/50
12/12 [=====] - 0s 4ms/step - loss: 11.3287 -
val_loss: 14.4121
Epoch 27/50
12/12 [=====] - 0s 5ms/step - loss: 11.3279 -
val_loss: 14.4134
Epoch 28/50
12/12 [=====] - 0s 5ms/step - loss: 11.3271 -
val_loss: 14.4139
Epoch 29/50
12/12 [=====] - 0s 3ms/step - loss: 11.3264 -
val_loss: 14.4145
Epoch 30/50
12/12 [=====] - 0s 5ms/step - loss: 11.3256 -
val_loss: 14.4154
Epoch 31/50
```

```
12/12 [=====] - 0s 4ms/step - loss: 11.3251 -  
val_loss: 14.4155  
Epoch 32/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3243 -  
val_loss: 14.4163  
Epoch 33/50  
12/12 [=====] - 0s 4ms/step - loss: 11.3238 -  
val_loss: 14.4172  
Epoch 34/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3233 -  
val_loss: 14.4174  
Epoch 35/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3228 -  
val_loss: 14.4190  
Epoch 36/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3223 -  
val_loss: 14.4194  
Epoch 37/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3218 -  
val_loss: 14.4196  
Epoch 38/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3213 -  
val_loss: 14.4186  
Epoch 39/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3208 -  
val_loss: 14.4175  
Epoch 40/50  
12/12 [=====] - 0s 4ms/step - loss: 11.3203 -  
val_loss: 14.4177  
Epoch 41/50  
12/12 [=====] - 0s 3ms/step - loss: 11.3197 -  
val_loss: 14.4182  
Epoch 42/50  
12/12 [=====] - 0s 4ms/step - loss: 11.3193 -  
val_loss: 14.4187  
Epoch 43/50  
12/12 [=====] - 0s 4ms/step - loss: 11.3188 -  
val_loss: 14.4193  
Epoch 44/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3183 -  
val_loss: 14.4207  
Epoch 45/50  
12/12 [=====] - 0s 6ms/step - loss: 11.3178 -  
val_loss: 14.4205  
Epoch 46/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3172 -  
val_loss: 14.4200  
Epoch 47/50  
12/12 [=====] - 0s 5ms/step - loss: 11.3168 -
```



```

val_loss: 14.4202
Epoch 48/50
12/12 [=====] - 0s 4ms/step - loss: 11.3164 -
val_loss: 14.4203
Epoch 49/50
12/12 [=====] - 0s 5ms/step - loss: 11.3162 -
val_loss: 14.4223
Epoch 50/50
12/12 [=====] - 0s 4ms/step - loss: 11.3158 -
val_loss: 14.4234

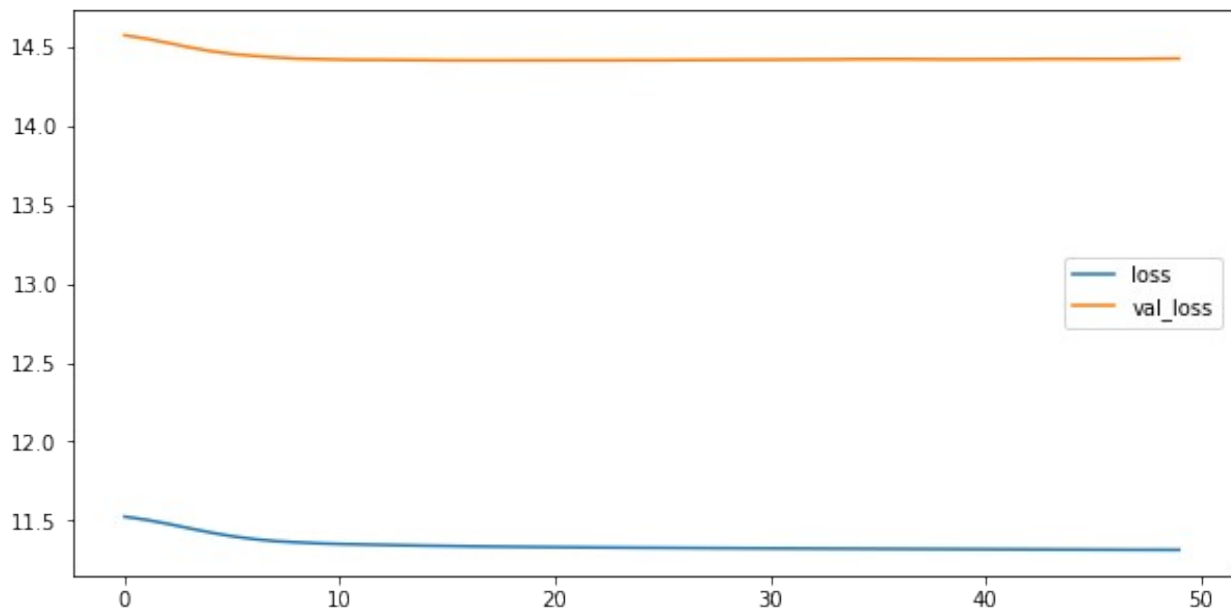
loss_df = pd.DataFrame(history.history)

# Plot loss vs epochs

loss_df.plot(figsize=(10,5))

<matplotlib.axes._subplots.AxesSubplot at 0x7f8c9778b290>

```



```

model.evaluate(test_data_prepared, test_label)

5/5 [=====] - 0s 3ms/step - loss: 18.7532
18.75318717956543

```

3.7 Saving and Loading a model

If something went well, from training to evaluating to improving a model, you would want to save it.

Here is how to save a model and how to load a saved model. The model will be saved in HDF5 format. When the model is saved in such format, the whole model things are saved.

```
model.save('forest_model.h5')
```

And loading the model is simple too...

```
loaded_model = keras.models.load_model('forest_model.h5')
```

You can make predictions on a loaded model.

3.8 Final Notes

It has been a quite long journey, from fitting a straight line to building a neural network for a real world dataset.

Ideally, for all datasets we used, neural networks would not be a suitable model. But because we are learning, it makes sense to start simple for the sake of understanding the latter.

In the next lab, we will do classification with neural networks. Later we will go deep into areas that neural networks have shown potential such as computer vision and natural language processing, and that's where we will practice all possible techniques of improving the results of the neural networks.

[BACK TO TOP](#)